University of Calabria
Department of Mathematics and Computer Science

Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Supervisors at the University of Potsdam:
Prof. Torsten Schaub
Prof. Javier Romero

Candidate:
Pierpaolo Spadafora

Student ID 263722

Academic Year 2025/2026

Contents

Abstract

Answer Set Programming is a declarative problem solving paradigm that lets a combinatorial problem be described declaratively and leaves the search of the solution to a specialized system. There are typically two types of approaches, ground-and-solve and lazy ground. In ground-and-solve systems, such as clingo, the relevant part of the program is instantiated before search starts, and this includes the domain-specific heuristics that tell the solver where to look first. Questo approccio è fortemente memory bound e questo limite si nota sia al crescere della dimensione delle istanze sia in casi dove le euristiche di dominio dipendono dallo stato della computazione, for instance through an aggregate over the atoms that are currently (currently as in "during the solving process") true, the grounder must then enumerate every value that state could take, and a single heuristic ends up creating a "combinatorially-big" impact on the memory. In lazy ground system both phases are interleaved allowing a bigger reach in "memory-intensive" problems at the cost of a slower solving time. This thesis presents the design, implementation and evaluation of a propagator to lazily evaluate heuristics in clingo. The propagator keeps domain-specific heuristics in a compact declarative form and evaluates them at solving time against the partial assignment, so that no `#heuristic` directive is ever expanded in more than one fact.

//TODO abbiamo usato due approcci per il backend, uno prolog, che ricalca l'apporccio utilizzato da quelli di vienna sincronizzando il solver tral aggiungendo e retraendo atomi nella KB di prolog e chiedendone la risoluzione nel decide e un approccio nativo in c++ con delle strutture dati specifiche per mantenere il conteggio degli aggregati.

The evaluation crosses two orthogonal axes: the *approach*, ground-and-solve against lazy ground, and the *interpretation of the negation*, clingo's semantics of a negated atom in a partial assignment against Alpha's; yielding the four variants `ga`, `gc`, `la` and `lc`, compared against a heuristic-free baseline on //TODO 3 common problms? Balanced Sum Problem (BSP), the Partner Units Problem (PUP) and the House Reconfiguration Problem (HRP). A technical note is the systematic *rewriting* of Alpha's weight@priority pairs into flattened clingo weights, //TODO che cazzo vuol dire? an order-preserving translation applied uniformly to every variant so that the heuristic remains a controlled constant while the two axes are varied.

// TODO queste sono delle conclusion più che un abstract, forse sarebbe sufficiente rimuoverlo da qui, no? The central experimental claim concerns memory and solving time. On BSP the number of ground heuristic objects produced by the ground-and-solve variants grows as $\Theta(n^3)$, reaching 1,010,200 ground `#heuristic` directives at $n = 100$, whereas the lazy variants keep exactly two heuristic facts, independent of n. On PUP the saving converts into reach: the unguided baseline and the plain ground heuristic stall by timeout at $N = 80$ under a fixed budget, while both Alpha-like variants solve every tested instance up to $N = 200$, the lazy one costing half the time and half the peak memory of its ground counterpart at each of the ten common sizes. On BSP, where the base program rather than the heuristic sets the frontier, the same mechanism instead reduces the search component of the run by three orders of magnitude, from 45 seconds to 0.04 at $n = 120$, while reaching the same wall. On HRP, whose heuristics contain no aggregates, the comparison

isolates the semantics of negation: the Alpha-like reading reproduces the intended guidance decision for decision, while the clingo-like reading is either inert or actively misleading. What laziness does not do is remove the cost of a dynamic heuristic. It moves that cost from grounding-time space to solving-time work, and the campaign quantifies the exchange through per-decision metrics of the propagator. An equivalence harness verifies throughout that all variants and both backends preserve the same answer sets, together with guards that detect silent misconfiguration of the Prolog backend.

1 Introduction and Background

1.1 The Cost of a Dynamic Heuristic

Answer Set Programming (ASP) is a declarative approach to combinatorial search. Instead of describing an algorithm step by step, the modeller describes the properties that an admissible solution must satisfy, and a solver enumerates the assignments that satisfy them [1, 2, 3] There are typically two types of approaches, ground-and-solve and lazy ground. In ground-and-solve systems, such as clingo, the relevant part of the program is instantiated before search starts, and this includes the domain-specific heuristics that tell the solver where to look first. Questo approccio è fortemente memory bound e questo limite si nota sia al crescere della dimensione delle istanze sia in casi dove le euristiche di dominio dipendono dallo stato della computazione, for instance through an aggregate over the atoms that are //TODO currently (currently as in "during the solving process") true, the grounder must then enumerate every value that state could take, and a single heuristic ends up creating a //TODO "combinatorially-big" impact on the memory. In lazy ground system both phases are interleaved allowing a bigger reach in "memory-intensive" problems at the cost of a slower solving time. //TODO Per dare una definizione un po' spartana e ortodossa, i solver sono dei motori ultra specializzati nel backtracking, molto efficienti nella risoluzione di problemi np-hard tramite un Domain Specific Language estremamente elegante. The cost of having this elegant and convenient DSL is control: the modeller defines *what* solution he wants, but not *how* the engine should look for it. A pure backtracking approach acts as a non informed search and in some cases where problems have very specific rules by cnstruction guiding the solver with information is extremeely helpful. Domain-specific heuristics //TODO sono il modo con cui il modellatore può utilizzare tali informaizoni per guidare la ricerca del solver. // TODO speculazione, non so se sono definite qui: In ASP core 2.0 le euristiche sono delle declarative annotations that gets grounded like normal atoms and their purpose is to tell the solver which atom and with which polarity to decide next; their impact is relevantly noticeable [4, 5]. They are, however, part of the program, and clingo instantiates the program before search starts. A heuristic is therefore expanded, in full, before the first decision is taken. That expansion can be relatively harmless as long as the heuristic depends only on the input. It stops being harmless as soon as the heuristic depends on *how the search is going*, which is precisely when a heuristic is most useful. Consider the directive used throughout this thesis to guide Balanced Set Partitioning, the problem of splitting $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` of nearly equal sum:

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
  ↪  true]
```

The rule reads: prefer to place the element X into b, the more strongly the larger the sum S of what is already in c.
It is a natural greedy policy, and it is dynamic//TODO citazione paper comploi taupe, because S is a property of the partial solution built so far, not of the instance. The aggregate value S appears *inside the weight*, and the atoms `c(Y)` it sums over are

guessed by the search rather than given as facts. The grounder cannot know what S will be. Its only option is to enumerate every value the sum could reach and to materialise a separate ground directive for every pair (X, S): on this encoding, $\Theta(n^3)$ objects, more than a million at $n = 100$, none of which carries any information the modeller wrote down. A //TODO stesso termine usato quando abbiamo descritto cosa sono le euristiche introduced to help the engine solve a problem quicker ends up putting answer out of reach.

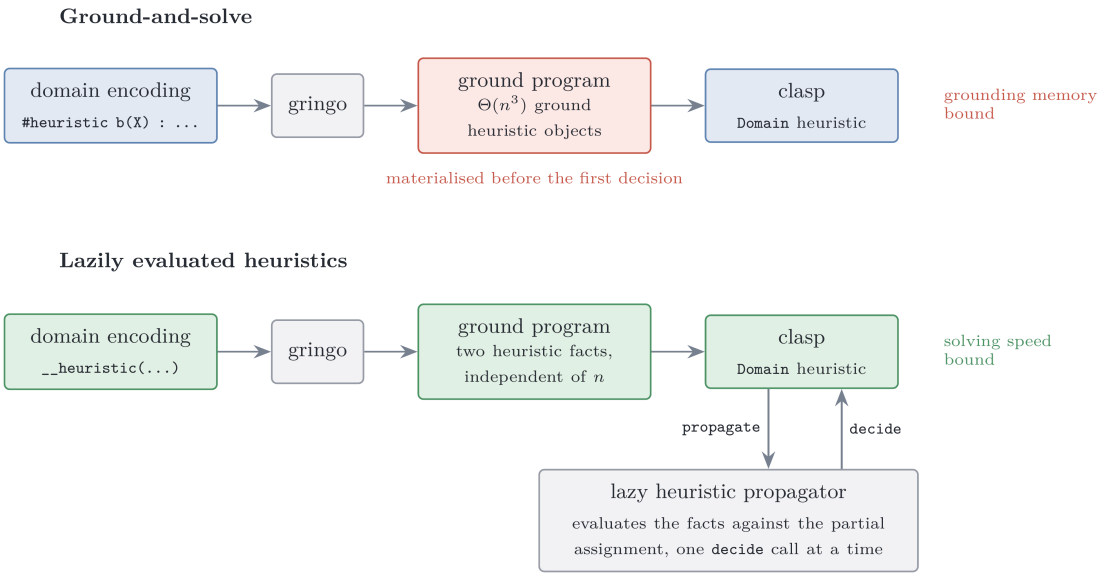

Figure 1: The same heuristic under the two approaches compared in this thesis.

The alternative is to leave the heuristic symbolic and to evaluate it while the search runs, when the partial assignment that S refers to actually exists. In Lazy-Grounding solvers this comes natural and it has already been tested [6] [7, 8]; This work wants to explore whether the same effect can be obtained inside clingo, without giving up its ground-and-solve pipeline de facto obtaining a ground-and-solve engine with lazily evaluated heuristics (Figure 1)

The rest of this section fixes the notions this work rests on: the semantics of ASP (Section 1.2), the ground-and-solve architecture and the search loop the propagator plugs into (Section 1.3), the `#heuristic` syntaxes and the two conventions of clingo that the thesis keeps returning to (Section 1.4), Alpha's reading of partial assignments, which is the semantic reference for the whole work (Section 1.5), and the interfaces through which clingo can be extended at solving time (Section 1.6). Section 1.7 then states the objective and the contributions precisely.

1.2 Answer Set Programming

A *normal rule* in ASP has the form

$$a \leftarrow b_1, \ldots, b_m,\ not\ c_1, \ldots, not\ c_n,$$

where a, the b_i and the c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j is. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c, not because c has been proven false.

The meaning of a program cannot be given by fixpoint iteration alone, it is given instead through the notion of a stable model. Fix a set of atoms X, the candidate solution, and a ground program P. The *reduct* P^X is obtained by deleting every rule whose negative body intersects X — those rules are the ones X itself invalidates — and by deleting the negative literals from the rules that survive, which X has already satisfied. The reduct is a positive program, so it has a unique least model, and X is a *stable model* (or *answer set*) of P exactly when it coincides with that least model [1]. The definition is a consistency test: X is stable when, assuming what X assumes to be false, one derives precisely X. On the two-rule program $\{\, a \leftarrow not\ b\,;\ b \leftarrow not\ a\,\}$ the candidate $X = \{a\}$ gives the reduct $\{a \leftarrow\}$, whose least model is $\{a\}$: stable. The candidate $\{a, b\}$ deletes both rules, leaving the least model $\emptyset$: not stable. Intuitively, then, a stable model is a set of atoms both closed under the rules and justified by them, in which every atom has a non-circular derivation that survives the model's own assumptions about what is false.

Problem solving in ASP follows the *guess-and-check* idiom: a choice construct opens the solution space, integrity constraints (rules with an empty head) prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. The input language of modern systems extends normal rules with first-order variables, arithmetic, choice rules such as `{ b(X) } :- x(X)`, conditional literals, and aggregate atoms such as `#sum`, `#count`, `#min` and `#max`, which condense a set of weighted contributions into a single comparable value [3]. Aggregates are central to what follows: the heuristics under study rank decisions by expressions such as "the current sum of the elements already placed in a set", and how and *when* such an aggregate is evaluated is exactly what separates the systems compared here.

1.3 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search

clingo [9] couples the grounder gringo with the solver clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free program; *intelligent instantiation* restricts this expansion to rule instances whose positive body is potentially derivable, so the ground program is in general much smaller than the full instantiation, but it must still be materialised in full before search starts. The solver clasp [10] then computes the stable models of the ground program with *conflict-driven nogood learning* (CDNL), the ASP counterpart of the CDCL algorithm that underlies modern SAT solvers [11]. The

solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates deterministic *unit propagation* with nondeterministic *decisions*, and when propagation derives a contradiction it analyses the conflict, learns a nogood that prevents its repetition, and *backjumps* to an earlier decision level. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; clasp's default is an activity-based scheme in the VSIDS family [11], which favours atoms that appear in recent conflicts.

Two features of this architecture matter for the thesis. First, every atom and every rule the solver will ever see must exist before the first decision is made: whatever the modeller wants the heuristic to consult at runtime has to be compiled, at grounding time, into propositional material. Second, the size of the ground program is the dominant memory cost of the pipeline, and when a program fragment multiplies value domains, its instantiation grows polynomially or worse in the instance parameters. This *grounding bottleneck* is a well-known limitation of ground-and-solve systems [7]; Section 3 measures one concrete manifestation of it, namely heuristic directives whose weight depends on an aggregate value.

1.4 Domain-Specific Heuristics in clingo

clasp exposes its decision heuristic to the modeller through the `Domain` heuristic and the `#heuristic` directive [4]:

```
1 #heuristic A : Body. [W@P, M]
```

For every ground instance whose *Body* holds, the directive attaches to the atom A a *modification* with weight W, priority P and modifier M. The modifiers refine different aspects of the decision: `sign` fixes the polarity with which A is decided, `level` places A in a ranking consulted before the default heuristic, `true` and `false` combine a level with a sign, and `init` and `factor` perturb the scores of the underlying activity-based heuristic rather than overriding it. Runs must select `--heuristic=Domain` for any of this to take effect.

Two conventions of this language are worth stating explicitly here, because the whole design of the thesis is organised around them. The first concerns *ranking*. The priority `@P` does *not* rank different atoms against each other: it arbitrates between multiple modifications targeting the *same* atom, the highest priority winning, while the global order in which distinct atoms are decided is induced by the resolved level value, that is, by the weight alone. The second concerns *truth*. The *Body* of a ground directive is evaluated with respect to *established* truth: a condition `not a` contributes only once a has been assigned false, and an aggregate has a value only once its source atoms are determined. Both conventions are natural in a pipeline that grounds first and searches afterwards, and both differ from the lazy-grounding reading described next. That pair of differences is what generates the experimental design of Section 2.

1.5 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Lazy grounding attacks the grounding bottleneck at its root, by interleaving grounding and solving: rule instances are produced during search, when their positive bodies become satisfied by the current assignment, instead of being materialised up front. Alpha [6] is the reference system of this family; it combines lazy grounding with a CDNL-style search loop, so it retains conflict learning while never holding the full ground program in memory.

Alpha's role in this thesis, however, is not that of a competing solver but that of a *semantic* reference. Comploi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics evaluated against the *partial* assignment maintained during solving [7]. Their semantics differs from clingo's on exactly the two points listed above. On truth, a negative heuristic condition *not a* is satisfied as soon as a is *not assigned true*, so a heuristic fires on undecided atoms instead of waiting for established falsity. On ranking, the annotations (W, P) form *global levels*: every applicable heuristic with priority P strictly precedes every heuristic with a lower priority, regardless of weights, and the weight discriminates only inside a level. A subsequent extension added *dynamic aggregates* to the heuristic language [8]: aggregate expressions in heuristic conditions are re-evaluated over the atoms currently true in the partial assignment, so their value evolves as search proceeds. This is what makes a heuristic such as "prefer the element whose insertion keeps the two sums balanced" directly expressible, and the two papers demonstrate the approach on configuration benchmarks, including the Partner Units Problem [12] and the House Reconfiguration Problem [13] used here.

The gap this thesis addresses is now visible. clingo offers a mature and efficient ground-and-solve pipeline whose heuristic conditions read established truth and whose aggregates are frozen at grounding time. Alpha offers heuristics that read the partial assignment and re-evaluate aggregates dynamically, but inside a different, lazily grounded system. Reproducing the second reading inside the first architecture, without paying the ground materialisation that a faithful ground simulation would require, is the subject of Sections 2 and 3.

1.6 Extending clingo: Propagators and the Heuristic Interface

Nothing in the previous sections suggests that clingo can be made to evaluate anything at search time: the pipeline grounds, then searches, and the searching half only ever sees propositional material. clingo does, however, expose a supported way in: *propagators*, user-supplied components registered with the solver that take part in the search loop [9]. The mechanism exists so that theories the ASP language cannot express natively — difference constraints, linear arithmetic, scheduling propagators — can be attached to clasp as external reasoners without forking it. This thesis uses the same door for a different purpose: not to add inference, but to add *guidance* that is computed rather than materialised.

A propagator is defined by four callbacks. During `init` it inspects the symbolic

atoms of the ground program, maps the atoms it cares about to solver literals, and registers *watches* on them. During search, `propagate` is invoked when a watched literal becomes true, `undo` when backtracking retracts it, and `check` on total assignments. The watches are what make the mechanism incremental and therefore affordable: the propagator is woken only on the trail changes it declared interesting, and can maintain its own state in lockstep with the solver's partial assignment rather than recomputing it.

The `Clingo::Heuristic` interface extends this contract with a `decide` callback, invoked at each decision point *before* the solver's built-in heuristic: it receives the current assignment and either returns a signed literal, which becomes the next decision, or returns 0, in which case clasp falls back to its default heuristic. Taken together, the two interfaces give exactly what the construction sketched in Section 1.1 needs, and nothing more: watches to observe the partial assignment as it evolves, and `decide` to act on what has been observed. The lazy heuristic mechanism of Section 2 is a heuristic propagator in this sense — it keeps the declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers each `decide` call with the best applicable target — and it is built entirely against the public API, without modifying clasp.

1.7 Objective and Contributions

The objective of this thesis is to design, implement and evaluate an extension of clingo that evaluates domain-specific heuristics lazily (or just lazy), and to compare it systematically against the ground-and-solve alternative. Because Section 1.4 and Section 1.5 isolated two independent differences between the two systems — *where* the heuristic is evaluated and *how* its conditions read partial information — the comparison is organised as a two-by-two design over those two axes, developed in Section 2.

Three questions structure the work.

1. **Representation.** How can a dynamic heuristic be expressed in a compact ground form that a propagator can interpret, and can the clingo-like and Alpha-like readings of partial information coexist in a single mechanism, selected per rule?

2. **Faithfulness.** How much of clingo's heuristic semantics — weights, priorities, modifiers — survives a reimplementation on top of the `Clingo::Heuristic` interface, and what does the mismatch between clingo's and Alpha's reading of priorities cost in encoding effort?

3. **Payoff.** What does the lazy representation actually buy, in grounding size, memory and search behaviour, on problems whose heuristics are dynamic in the sense of Section 1.1?

The implemented system consists of two modified clingo builds sharing the same source layout. The build in `clingo-native` carries the native backend: shared types, a parser for `__heuristic` facts, incremental aggregate states and the heuristic propagator, all under `libclingo`. The build in `clingo-prolog` carries the Prolog

backend: the same propagator skeleton, a rule-string normaliser, an in-process SWI-Prolog evaluation backend [14] and an auxiliary clingo-based fallback evaluator. In both builds the propagator is registered automatically in `clingo_app.cc`, so the modified binaries use the mechanism without any external controller. The two backends embody two different engineering answers to the same question — compiled incremental evaluation against a general logic-programming runtime, over the same declarative content — and comparing them is part of the evaluation.

On the experimental side, the project provides encodings of three benchmark problems — Balanced Set Partitioning (BSP), the Partner Units Problem (PUP) [12] and the House Reconfiguration Problem (HRP) [13] — in every cell of the design and for both backends, under `test_folder/encodings-native` and `test_folder/encodings-prolog`; an equivalence harness, `tools/check_equivalence.py`, which verifies that the heuristics preserve the answer sets and that the Prolog backend is actually active; and a benchmark pipeline built on `benchmark-tool` and `runlim` [15], with wrapper systems for the two binaries, a result parser that extracts solver and propagator statistics, and automatic graph generation.

//TODO Link al codice sorgente: https://github.com/pierspad/

!

2 Methodology

2.1 The Experimental Design: Two Axes, Four Variants

Every experiment in this thesis compares encodings of the same problem that differ *only* in how the domain-specific heuristic is represented and interpreted. The domain logic is held fixed; what varies are the two properties that Section 1.4 and Section 1.5 identified as the points on which clingo and Alpha disagree, and they vary independently.

The *approach* axis fixes where the heuristic is evaluated. In the ground-and-solve approach the heuristic is written with native `#heuristic` directives: gringo expands them before search and clasp's Domain heuristic consumes the resulting ground objects. In the lazy approach the heuristic remains a handful of ground facts, which the propagator reads at initialisation and evaluates at every decision point against the partial assignment.

The *semantics* axis fixes how a heuristic condition reads partial information: the clingo-like reading, under which `not a` holds only once a has been assigned false and an aggregate has a value only once its sources are determined, against the Alpha-like reading, under which `not a` holds as soon as a is not assigned true and an aggregate is computed over the atoms currently true. Only the second reading makes a heuristic dynamic in the sense of Section 1.1: an instruction such as "prefer the element whose insertion keeps the sums balanced" presupposes that the sum can be read off an assignment that is still being built, and under the clingo reading that sum simply has no value until it is too late for the guidance to matter.

Crossing the two axes yields the four variants used throughout, plus the heuristic-free baseline `gc_noheur`, which carries the same domain logic with no heuristic at all (Figure 2).

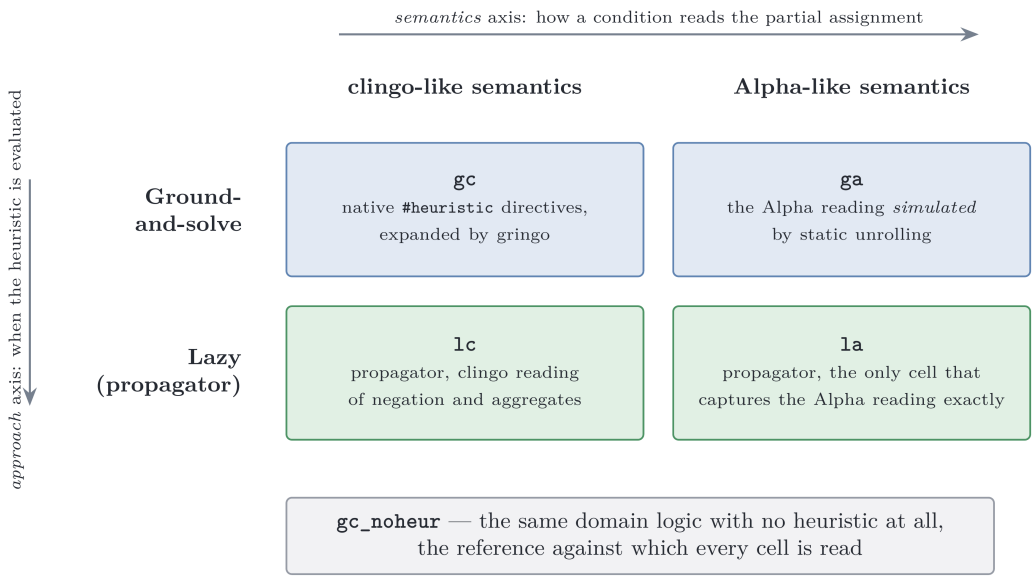

Figure 2: The experimental matrix. Each cell exists for both evaluation backends (native and Prolog) and for each benchmark problem (BSP, PUP, HRP). Prefixes name the approach, suffixes the semantics, so a difference between `la` and `lc` is attributable to the semantics and a difference between `la` and `ga` to the approach.

Two structural constraints shape the matrix, and both are consequences of the architecture rather than choices. First, ground clingo cannot express the Alpha semantics exactly: there is no ground construct denoting "the sum of the atoms currently true". The `ga` variants therefore *simulate* the Alpha reading, in the precise sense described in Section 2.1, and only `la` captures it exactly. Second, the pair `gc`/`ga` is kept symmetric to the pair `lc`/`la`: within each approach the two semantics differ exclusively in the treatment of negation and aggregates, and nothing else in the encoding is allowed to move.

Simulating Alpha in Ground clingo

The `ga` encodings apply two systematic transformations to `gc`.

The first transformation drops the negative literals that test the partial state (for BSP, `not c(X)` and `not b(X)`; for PUP, the `not not_assigned_...` conditions; for HRP, guards such as `not fullCabinet(C)`). Under the Alpha reading these literals are satisfied on undecided atoms as well as on false ones, and an undecided atom is the common case at the moment a heuristic fires; removing the literal altogether is the closest ground approximation, and it is harmless in practice because the solver never re-decides an atom that is already assigned.

The second transformation replaces exact aggregate bindings by *bounds over a static domain*. For BSP:

```
sum_value(0..tot).
#heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

Instead of binding S to the final value of the sum, the directive is unrolled over all candidate values `sum_value(S)` and guarded by the monotone condition $S \leq \#\texttt{sum}\{Y : c(Y)\}$. During search, as soon as the partial sum reaches a bound S, every directive with that bound becomes applicable; among the applicable ground directives for the same atom, clasp's Domain heuristic retains the one with the maximum weight, which corresponds exactly to the *current* value of the growing aggregate. The max-weight selection thus reconstructs the dynamic aggregate at solving time. For aggregates whose contribution to the weight decreases rather than increases (the free-capacity terms N_0, N_1, N_2 in PUP) the same trick is applied with the dual bound $V \geq \#\texttt{max}\{\ldots\}$, so that the max-weight selection picks the smallest proven value.

The simulation is faithful, then, but its price is exactly the combinatorial explosion that motivated the thesis: one ground directive per point of the product of the unrolled domains, measured in Section 3.

Weights, Priorities, and Flattening

Section 1.4 and Section 1.5 recorded that the two systems read the annotation $[W@P]$ differently: in Alpha the pair (W, P) ranks all heuristics globally, whereas in clingo `@P` arbitrates only between modifications of the same atom, and the global order comes from the weight alone. That difference is not a detail of notation. It means that an encoding written for Alpha cannot be transplanted into clingo verbatim: any ordering intended *between* different atom families (zones before sensors in PUP; reuse before cabinet-filling before room-filling in HRP) is simply lost, because clingo has no field that expresses it.

The intended order can nevertheless be recovered, by *flattening* the level into the weight:

$$W_{\mathit{flat}} \;=\; W + P \cdot M, \qquad M > \max W + 1,$$

where M is a level multiplier strictly larger than any intra-level weight, so that no heuristic of a lower level can outrank a heuristic of a higher one. In the PUP encodings M is the constant offset 100,000 added to the zone weights, against a maximum sensor weight of 46,220 on the benchmark instances; in HRP M is derived from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers.

Flattening is order-preserving, and this is what makes it an admissible translation rather than an approximation. If M exceeds every intra-level weight, then $W + P \cdot M > W' + P' \cdot M$ holds exactly when (P, W) precedes (P', W') lexicographically, so the total order induced on the atoms is the one Alpha's absolute levels induce. Nothing of the strict discrimination between levels is lost; only its carrier changes, from a separate field to the weight itself.

The suite therefore adopts a single convention, applied uniformly to *all four variants and both backends*: absolute levels always live in the weight, and the priority field is reserved for its local role, arbitrating between several directives attached to the *same* atom, exactly as `@P` does in clingo. In BSP the priority is 0 throughout, since no two directives compete on one atom; in PUP likewise; in HRP it takes the two values 1 and 0, enough to let the `true` directives win over the closing `false`

directives on the same target, which is the role clingo's implicit default $P = |W|$ plays in the ground variants.

This is a deliberate methodological choice, not a limitation of the mechanism. The native propagator does implement Alpha's absolute levels natively: under `__semantics(alpha)` it ranks candidates globally by (priority, weight), and the C++ test suite covers that behaviour together with its local-only counterpart under `__semantics(clingo)`. The benchmark encodings do not exercise it because the experiment requires the heuristic to be a controlled constant: if `la` expressed the intended order through native levels while `gc`, `ga` and `lc` expressed it through flattened weights, a difference in the results could no longer be attributed to the axes under study. Since ground clingo cannot rank different atoms by priority at all, flattening is the only mechanism available in every cell of the design, and hence the only one that keeps the four variants comparable. The same argument holds across backends: the Prolog backend sorts globally by (priority, weight) irrespective of the semantics selector, so an encoding carrying its levels in the priority field would not mean the same thing in the two builds. Under the convention, every pair of encodings differs by exactly one token — `__semantics(alpha)` against `__semantics(clingo)` in the native DSL, `alpha_not` against `clingo_not` in the Prolog rules.

A last convention concerns empty aggregates, where the two systems again disagree silently: `#max` over an empty set is 0 in Alpha but $\#inf$ in clingo. All encodings therefore write `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same rule (the native `MaxState` returns 0 when empty; the Prolog runtime defines `dyn_max` with a 0 default).

2.2 System Architecture

The project maintains two modified copies of clingo with the same source layout. The build in `clingo-native` implements the native backend; the build in `clingo-prolog` implements the Prolog backend and is configured with `CLINGO_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process. In both builds the extension lives entirely under `libclingo` and is registered in `clingo_app.cc`:

```
1 HeuristicPropagator my_propagator;
2 ctl.register_propagator(my_propagator, true);
```

Those two lines are the whole integration: everything downstream is a consequence of clasp calling back into the class, and no external Python or C++ controller is involved. Figure 3 shows where the class sits.

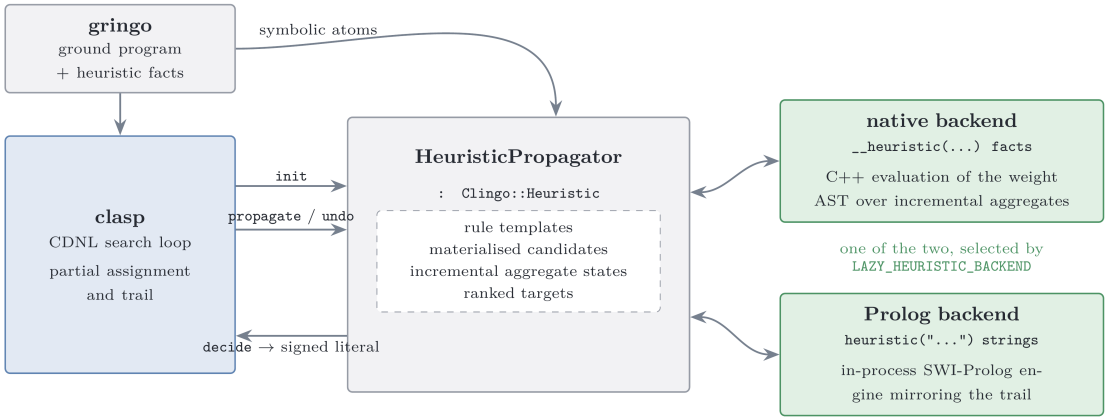

Figure 3: The heuristic propagator between clasp and the two evaluation backends. The propagator receives the ground program's symbolic atoms at `init`, follows the trail through `propagate` and `undo`, and answers `decide` with a signed literal. The backend that performs the evaluation is chosen at run time and is invisible to the solver.

The class implements `Clingo::Heuristic`, that is, the propagator interface extended with the `decide` callback of Section 1.6. Both backends follow the same lifecycle. During `init` they scan the symbolic atoms for their trigger facts — `__heuristic(...)` in the native build, `heuristic("...")` in the Prolog build — build their runtime representation, and register watches on the solver literals they need to observe. During search, `propagate` and `undo` keep the internal state synchronised with the trail. At each decision point, `decide` either returns a signed literal for the best applicable heuristic target or returns 0, leaving the decision to clasp's fallback heuristic. Runs are launched with `--heuristic=Domain`.

One property of this design deserves to be stated plainly, because everything the experiments claim depends on it: the propagator never adds a constraint, never derives a literal and never refuses one. It only reorders the decisions clasp would have taken anyway, so the answer sets are untouched, and a heuristic can make a run slower or faster but cannot make it wrong. Section 2.6 turns this from an argument into a check.

2.3 The Native Backend

From a `#heuristic` Directive to a Lazy Heuristic

The native backend replaces a family of ground directives with a single ground fact that describes them. The description has to be complete enough for the propagator to reconstruct, at run time, what gringo would have written out — and small enough that the grounder has nothing to expand. This subsection derives that fact for the BSP heuristic of Section 1.1, one component at a time; the other encodings are built the same way.

The starting point is the directive as clingo reads it:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
↪  true]
```

The target. The atom being guided is `b(X)`, where X ranges over the ground instances. The lazy fact names only the *predicate*, `__target(b)`; the instances are found by the propagator at `init`, when the ground program is available and every `b(...)` atom in it can be enumerated for free. This is the first place where the grounding work disappears: what was a per-instance directive becomes a per-predicate template.

The positive body. The condition `x(X)` shares its argument with the target, so it is a filter on the target tuple rather than a join: the candidate for `b(7)` is alive only while `x(7)` is true. Such conditions are written as the bare predicate name, `x`, and the propagator matches them positionally against the target tuple.

The negative body. The condition `not c(X)` likewise shares the target tuple, and is written `__n_c`, the prefix marking negation. What that literal *means* is deliberately not encoded here: it is the semantics selector, further down the fact, that decides whether it is read as "`c(X)` is assigned false" or as "`c(X)` is not assigned true". This is what allows a single fact to be turned from an `la` heuristic into an `lc` heuristic by changing one token.

The aggregate. The subexpression $S = \#\mathtt{sum}\{Y : c(Y)\}$ is the part that ground clingo cannot keep symbolic, and it is written `__bind(s, __sum(c))`: it introduces a run-time variable `s`, bound to the sum over the source predicate `c`, maintained incrementally as literals of `c` enter and leave the trail. Nothing is enumerated: where the ground directive needed one copy of itself per reachable value of S, the fact holds a name and a rule for keeping its value current.

The weight. The arithmetic $((n + 1) \cdot S) + X$ becomes the expression tree `__weight(__add(__mul(s, B), self))`, built from `__add`, `__sub` and `__mul` over constants, bound variables and the reserved symbol `self`, which denotes the first numeric argument of the target — here the X of `b(X)`. The tree is evaluated per candidate at decision time, with `s` holding whatever the sum currently is.

The remaining fields. `__priority(0)` carries the priority, in its local role as fixed by Section 2.1; `__modifier(true)` carries the clingo modifier unchanged; and `__semantics(alpha)` selects the reading of negation and aggregates. Assembling the components gives the fact in full:

```
__heuristic(__target(b), x, __n_c, __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)), __priority(0),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

The ASP body `:- B = n + 1` is a small but useful idiom. The fact is ground either way; the body merely lets gringo evaluate the program constant `n` and inject $n+1$ as a plain number, so that the propagator never has to know what a program constant is. The same heuristic in the Prolog backend is the rule string of Section 2.4, and Table 1 lines up the three formulations construct by construct.

clingo directive	**native lazy fact**	**Prolog rule string**
target `b(X)`	`__target(b)`	head `heuristic(b(X),W,P,M)`
positive body on the target tuple, `x(X)`	bare predicate `x`	`holds(x(X))`
positive body with its own arguments	`__body(pred, __match(...), __bind_arg(...))`	any Prolog goal over `holds/1`
negative body `not c(X)`	`__n_c`	`alpha_not(c(X))` or `clingo_not(c(X))`
aggregate `S = #sum{Y : c(Y)}`	`__bind(s, __sum(c))`	`aggregate_all(sum(Y), holds(c(Y)), S)`
weight `W = ((n+1)*S)+X`	`__weight(__add(__mul(s,B), self))`	`W is Base*S+X`
priority `@P`	`__priority(0)`	third head argument
modifier `[W, true]`	`__modifier(true)`	fourth head argument
— (fixed by the system)	`__semantics(alpha | clingo)`	choice of `alpha_not` / `clingo_not`
— (implicit in clasp)	implicit: candidates with an assigned target are skipped	`target_available(b(X))`

Table 1: The same heuristic in the three notations. The lazy formulations are per-predicate templates, so their size does not depend on the instance, whereas the clingo directive is per ground instance.

The Rest of the Syntax

Not every heuristic is as compact as BSP's, and the DSL grows in the two directions the benchmark encodings actually need.

When a positive body predicate does *not* share the target tuple, the correspondence has to be given explicitly:

```
__body(pred, __match(src, tgt), __bind_arg(var, idx))
```

which states which argument of the body atom must equal which argument of the target, and which further arguments are to be extracted into run-time variables usable in the weight; `__bind_target_arg(var, idx)` does the same for an argument of the target itself. These are the constructs that turn a template into a join, and they are the reason candidate materialisation (Section 2.3) can produce more than one candidate per target.

Aggregates, in turn, support per-target filtering. In PUP one needs the maximum number of sensors on the *previous* unit, which is written

```
__bind(m1, __max(num_sensors_on_unit, 0, __filter(1,1,-1)))
```

and reads, for a target `assigned_zone_unit(Z,U)`: maintain the maximum of the first argument of `num_sensors_on_unit(N,U')` over the source atoms whose second argument satisfies $U' = U - 1$, with 0 as the value on the empty set. The triple in `__filter` is (source index, target index, offset), so a filter is a linear relation between one source argument and one target argument.

One restriction is worth declaring: the aggregate machinery draws from a single source predicate, so joins occurring inside an Alpha-style aggregate cannot be expressed directly. Where the original encodings need one, it is precompiled into an auxiliary derived predicate — for PUP, `sensor_assigned_zone/2` and `sensor_zone_on_unit/3` — which the aggregate then reads as a single source. These auxiliaries are pure definitions that appear in no constraint, so they cannot change the answer sets; they do add grounding, and that cost is charged to every variant alike, so it cannot flatter the lazy ones.

Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target predicate, positive body specifications, negative body predicates, variable bindings, modifier, semantics, and two arithmetic ASTs for weight and priority. Malformed syntax, duplicate variables, unknown operators and negative indices are rejected eagerly with specific messages. This is a deliberate stance rather than defensive programming: a heuristic that fails to parse is not an error clasp can notice, since a program with no heuristic is a perfectly valid program, so a misspelled directive would otherwise degrade silently into an unguided run that still produces correct answers — and, in a benchmark, into a data point that means nothing.

What happens next is where the laziness becomes concrete. During `init` the propagator receives the symbolic atoms of the ground program and builds an index from predicate names to the solver literals of their numeric tuples. It then *materialises candidates*: for every ground target atom, and for every combination of matching positive body atoms, it creates a `RuntimeHeuristicCandidate` holding the target literal, the target tuple, the body literals, the values bound from body arguments and the instantiated aggregate keys. On BSP at $n = 100$ this produces one hundred candidates, one per `b(X)`; the corresponding ground encoding produces 1,010,200 directives, because it must also enumerate the values of S. That is the entire difference: candidate materialisation is proportional to the number of ground targets, and it does *not* multiply with the value domains of the aggregates, because an aggregate is now a variable to be read rather than a value to be enumerated. It also happens inside the propagator's memory, as small POD structures, rather than as printed ground directives carrying symbolic weights.

Watches are registered last, and only where a change can matter: on both polarities of the target and of the negative body literals, and on the positive body and aggregate source literals. Everything else in the program can move without waking the propagator.

Incremental Aggregates

Aggregate states implement a minimal interface — `add`, `remove`, `result` — and the choice of data structure follows from the fact that search backtracks. Sums and counts are plain accumulators, since addition has an inverse. Minima and maxima do not, so they are ordered multisets rather than a single running value: after `remove`, the state must return the maximum of what is left, which a scalar cannot reconstruct. Each source atom contributes its numeric value to the `RuntimeAggregateKey` obtained by instantiating the filter values; contributions are applied in `propagate` when a source literal becomes true, and reverted in `undo`, so the state is always the aggregate of exactly the atoms currently on the trail.

Aggregates read under the clingo semantics need one more thing, and this is where the semantics selector reaches into the data structures. Since a clingo-like aggregate has a value only when its sources are determined, the propagator tracks the full set of source literals per runtime key and *gates* the aggregate: it is usable only once every source is assigned, and until then the candidate that depends on it is simply not active. Under the Alpha semantics no gate exists and the current value is always usable — which, on the benchmarks with aggregate-bearing heuristics, is the whole reason the `lc` variants behave as they do.

Evaluation and Decision

A candidate is *active* when its target is still free, all its positive body literals are true, and every negative body literal is satisfied according to the template's semantics — the single place where the two readings of negation differ, and small enough to quote in full:

```
static bool negative_literal_is_satisfied(HeuristicSemantics s,
    Clingo::TruthValue v) {
    return s == HeuristicSemantics::Clingo ? v == Clingo::TruthValue::False
                                           : v != Clingo::TruthValue::True;
}
```

An active candidate must also be able to build its variable environment, which for clingo-semantics templates includes the aggregate gating described above. Its effects are then folded per target into a `TargetHeuristicState`. The modifiers follow clingo's Domain heuristic: `level` sets the ranking value, `sign` the preferred polarity, `true` and `false` set both. `init` and `factor` are recognised but rejected, because they perturb solver-internal activity scores that the `decide` interface cannot reach; mapping them onto `level` would silently turn a nudge into an override, which is worse than refusing them.

The per-target resolution and the global ranking are where the two readings of priority from Section 2.1 are actually implemented. For clingo-semantics effects the local priority arbitrates between modifications of the same target, and the target then enters the global ranking with its resolved weight only. For Alpha-semantics effects the best effect per target is selected by (priority, weight), and the pair itself becomes the global rank of the target, reproducing Alpha's levels. Ranked targets live in an ordered set:

```
std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

ordered by decreasing priority, then decreasing weight, then a deterministic literal tie-break, so that two runs of the same encoding take the same decisions. `decide` pops stale entries lazily — targets that have since been assigned, or whose state changed after they were ranked — applies the resolved sign to the best valid target, and otherwise returns 0 and lets clasp choose. `undo` uses `noexcept` refresh paths throughout, since an exception thrown while the solver is backtracking would leave it in a state it has no way to repair.

2.4 The Prolog Backend

The native backend buys its speed by compiling the heuristic language into C++ data structures, which fixes what the language can say. The Prolog backend takes the opposite position: it accepts an actual logic program as the heuristic and evaluates it with a general-purpose engine, so the expressiveness question disappears and the cost question takes its place. The two are interchangeable at run time and share the whole propagator skeleton, which makes the comparison in Section 3.13 a comparison of evaluation strategies and of nothing else.

Rule Strings and Normalisation

In the Prolog backend a heuristic is a rule string whose head is a four-argument term carrying target, weight, priority and modifier. The BSP heuristic of Section 2.3 reads:

```
heuristic("heuristic(b(X), W, 0, true) :-
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    W is Base*S+X.").
```

The body is genuine Prolog, and the correspondence with the native fact is the one tabulated above: `holds/1` for a positive condition, `alpha_not/1` for a negative one, `aggregate_all/3` for the dynamic sum, an `is/2` for the weight, and `target_available/1` for the requirement that the target still be undecided — which in the native backend is implicit in candidate selection but here has to be said, since a Prolog rule will happily propose an atom that has already been decided.

The propagator collects all `heuristic/1` facts (the alias `prolog_heuristic/1` is accepted for compatibility), normalises each string, and classifies its semantics syntactically: a rule containing `clingo_not(...)` is clingo-like, otherwise it is Alpha-like. To keep the synchronised state small it then extracts, statically, the predicate signatures that actually occur in the rules — inside `holds`, `alpha_not`, `clingo_not`, `target_available`, and as rule targets — and mirrors only atoms with those signatures. Without this filter the backend would assert the entire ground program into the Prolog database at every decision level, which on the larger instances dominates everything else.

The Runtime Program

At initialisation the backend boots the SWI-Prolog engine in-process [14] and consults a generated runtime program that defines the evaluation vocabulary:

```
holds(A) :- true_atom(A).
holds(A) :- static_atom(A), \+ true_atom(A).
clingo_not(A) :- false_atom(A).
alpha_not(A)  :- \+ holds(A).
target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).
dyn_max(Goal, Template, Max) :-
    (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).
```

These few clauses are the entire semantic bridge, and each of them is one of the conventions fixed earlier. The solver trail is mirrored by the dynamic predicates `true_atom/1` and `false_atom/1`, and a free atom is represented by absence from both — so a three-valued assignment is encoded in two two-valued predicates, and `\+ holds(A)` means "false or undecided", which is exactly Alpha's reading of negation. `clingo_not` instead demands explicit falsity. `aggregate_all` over `holds` computes an aggregate on the atoms currently true, which is Alpha's aggregate semantics obtained for free from Prolog's own evaluation order. `dyn_max` and `dyn_min` add the empty-set-is-zero convention of Section 2.1, and `target_available` enforces the requirement discussed above.

Synchronisation and Decision

Trail synchronisation is incremental: `propagate` and `undo` touch only the atoms mapped by the literals that changed, each with a single combined retract-and-assert goal, because in this backend term parsing and calling — not the logical work — dominate the cost of a synchronisation. At each `decide` the backend enumerates the solutions of `heuristic(T, W, P, M)`, discards candidates whose target is assigned or unmapped, and selects the best by decreasing priority, then decreasing weight, then a deterministic tie-break, with the modifier giving the sign of the returned literal. Note that this is a full re-enumeration per decision, where the native backend maintains a ranked set incrementally; the per-decision instrumentation of Section 3.7 is designed to expose precisely that difference.

The backend is enabled by setting the environment variable `LAZY_HEURISTIC_BACKEND` to `prolog`, and with `LAZY_PROLOG_STATS` set to 1 it prints a summary line with decision counts and cumulative timings of every phase (state synchronisation, Prolog query, candidate scan, selection), which the benchmark parser ingests.

Without the environment variable the same build falls back to an auxiliary evaluator that re-grounds a small clingo program per decision. That fallback accepts a different, ASP-style body syntax, so a Prolog-style rule handed to it simply fails — SWI's `unknown=fail` convention — and the run degenerates to no heuristic at all while still terminating successfully and still producing correct answer sets. The failure is invisible in exit codes and in output, and it is therefore exactly the kind of misconfiguration that quietly corrupts a benchmark campaign; the guards that detect it are described in Section 2.6.

2.5 Benchmark Problems and Encodings

Balanced Set Partitioning (BSP)

BSP partitions $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` whose sums differ by at most one. The guessing part is a symmetric even loop, the check compares the two sums, and the heuristic prefers to place the next element into the set whose current sum is largest, keeping the partition balanced greedily. It is the smallest problem in the suite and the one whose heuristic is purest: a single aggregate, no ordering between atom families, nothing else moving.

That purity has a consequence worth stating, because it is easy to misread the weight. The flattening rule of Section 2.1 is *vacuous* on BSP: all its directives belong to one level, since `b/1` and `c/1` are ranked by one and the same criterion. The weight $((n+1) \cdot S) + X$ is therefore not a flattened weight–priority pair but the lexicographic composition of the two components of a single criterion — the dynamic sum S dominating, the element index X breaking ties inside it — with $n+1$ playing the role of the multiplier *within* the criterion rather than between levels. Accordingly all four variants carry priority 0, `la` included, and the `la`/`lc` contrast on BSP isolates the semantics axis alone, with no interference from the priority reading.

Besides the four matrix variants and the baseline, BSP carries three methodological extras: `ga_weak`, which drops the negative literals without the static unrolling and therefore changes only the moment at which the heuristic activates; `la_aux`, which routes the lazy heuristic through an auxiliary predicate `h_b(X,S)` that reintroduces grounded aggregate values, and hence a grounding cost, before the propagator is even reached; and `la_co`, which combines the lazy heuristic with a linear reformulation of the balance constraint, and is treated as a modelling experiment rather than a heuristic comparison.

Partner Units Problem (PUP)

PUP [12] connects zones and sensors to communication units under adjacency and capacity constraints; the instances of the `double` family provide `comUnit/1` and `zone2sensor/2`. The encoding follows the Alpha evaluation encoding with an inductive, locality-based choice structure. The heuristic ranks sensor placements by a four-component weight (dynamic degree, forbidden placements, unit occupancy, direct connections) and zone placements by occupancy and free capacity, with zones globally preceding sensors — so PUP is the benchmark on which the flattening of Section 2.1 does real work.

Porting the encoding from Alpha to clingo required four adaptations, each forced by the difference between lazy and eager grounding, and each applied uniformly to *all* variants so that no comparison is affected:

- the inductive `assignable` rules are bounded by `comUnit(U±1)`, because gringo would otherwise materialise an unbounded chain that Alpha explores lazily;
- the existential capacity constraints are rewritten as monotone counting aggregates with linear grounding;

- a redundant distance-two blocking rule with cubic grounding is removed, since it prunes nothing the remaining constraints do not already prune;
- counters are capped at the unit capacity (`sensorNumbers(0..2)`), without which the auxiliary predicate `num_forbidden_places_of_sensors` explodes in every variant and masks the effect under study.

The zones-before-sensors ordering is flattened into the weight in all four variants and in both backends (offset +100,000, against a maximum sensor weight of 46,220), with the priority field left at 0 throughout. The related `doubleV` family is not used here; the reasons are taken up with the other limitations in Section 4.

House Reconfiguration Problem (HRP)

HRP [13] extends the House Configuration Problem with legacy configurations: things must be stored in cabinets and cabinets placed in rooms owned by the things' owners, under capacity constraints refined by the thing-length/cabinet-size extension, while a legacy configuration can be partially reused or dismantled. The heuristic is a strict five-level policy: reuse legacy components first (with internal weights), then fill cabinets with long things, then with short things, then place cabinets into rooms, and finally a closing level that biases all remaining choice atoms to false, pruning the tail of the search.

HRP is the negation-heavy benchmark, and that is why it is in the suite: its heuristics contain no aggregates at all, so the `la`/`lc` comparison isolates the effect of the negation semantics alone, whereas in BSP and PUP the two mechanisms act together and cannot be told apart. In the native lazy encodings the guards over the partial state are precompiled into same-tuple auxiliary predicates — for example `cabFull(C,T) :- cabinetDomain(C), thing(T), fullCabinet(C)` — so that negative conditions align with the target tuple as the DSL requires. The five levels are flattened with the instance-derived multiplier `levelMul(LM) :- LM = MC + MR + 10` in all four variants and in both backends, and the priority field retains only its local role, distinguishing the `true` directives from the closing `false` ones on the same target.

2.6 Correctness Validation

Section 2.2 argued that a heuristic cannot change the answer sets. The argument is sound but it is still an argument about code, and the claim it supports is falsifiable, so the pipeline checks it before measuring anything.

The harness `tools/check_equivalence.py`, run by the sanity entry point ahead of every campaign, performs an *exhaustive equivalence* check on a small BSP instance: it enumerates all models with `clingo -n 0` for every variant and both backends, projects the models onto the solution predicates `b/1` and `c/1`, and requires the projected collections to be identical across variants and across backends. The projection is not cosmetic: the encodings expose different `#show` sets and carry different auxiliary predicates, so comparing raw witnesses would report differences that are artefacts of the instrumentation. For PUP, where exhaustive enumeration is intractable even on small instances, the harness falls back to agreement on SAT/UNSAT together with the choice counts.

The second thing the harness guards against is the silent-fallback failure mode of the Prolog backend described in Section 2.4, which is more dangerous than an outright bug precisely because it produces correct results. Every lazy Prolog run must report `decide_calls > 0` in the propagator summary: a run that terminated successfully but never called the custom `decide` has degenerated to the no-heuristic baseline, and is treated as an error by the harness and flagged with an explicit warning column by the benchmark runner. As a cross-check, the harness verifies that native `la` and Prolog `la` perform the same number of choices on the control instance — if the SWI engine were inactive, the two runs would diverge immediately. The benchmark wrapper exports `LAZY_HEURISTIC_BACKEND=prolog` itself, so the guard protects against configuration drift rather than repairing it.

The C++ unit tests complete the picture on the native side, covering incremental aggregates, corner cases (empty domains, unsatisfiable instances), explicit body and target bindings, filtered aggregates, the local-versus-global reading of priorities in the two semantics, and the syntax validation of the parser.

2.7 Benchmark Infrastructure and Metrics

The campaigns are orchestrated with `benchmark-tool`, driving one run per (system, setting, instance) triple under `runlim` [15] for wall-clock and memory limits. The two systems are shell wrappers around the two modified binaries; both fix `--stats=2 --heuristic=Domain -n 1` and a numeric locale, and the Prolog wrapper additionally exports the backend activation and statistics variables. A custom result parser extracts, for every run: outcome and times (total, solving, and their difference as an estimate of the non-solving part), search statistics (choices, conflicts, restarts), grounding-size measures (ground `#heuristic` directives for the ground variants, lazy heuristic facts for the lazy ones, and rule counts from clasp's statistics), the peak memory, and, for both lazy backends, the propagator summary (decision calls, candidates seen per decision, cumulative synchronisation, query, scan and selection times). The two backends emit that summary through the same parser, so the per-decision cost of the compiled and of the interpreted evaluation are directly comparable; the one metric that is *not* comparable across them is the candidate count, since the native backend reports the candidates it retains after incremental filtering while the Prolog backend reports every solution the `heuristic/4` query returns. Wherever a candidate count appears in Section 3 the backend it comes from is named.

Three measurement caveats are declared up front, because they define what the numbers can and cannot show. First, the campaigns run on a shared cluster whose nodes are not identical, and SLURM dispatches each job to whichever node is free. Wall-clock times therefore carry a node-to-node spread that Section 3.2 quantifies on programs known to be identical: a median of 13% and a maximum of 52% on the grounding component. The structural counters emitted by clasp and by the propagator — choices, conflicts, rules, atoms, variables, decide calls — are unaffected, and the analysis leans on them wherever a time gap is not an order of magnitude wide. Second, the memory metric is the peak resident set size of the whole solver process, end to end: for the lazy Prolog variants it includes the in-process

SWI engine and its asserted database. The engine has a fixed initialisation overhead independent of n, so on small instances the lazy variants may show a *larger* RSS than the ground ones; the lazy memory advantage is asymptotic, emerging where the $\Theta(n^3)$ growth of ground heuristics overtakes that constant. Being a property of the program rather than of the machine, however, peak RSS does not suffer from the first caveat, which is why it carries the central memory claim of Section 3.11. Third, the ground-size comparison counts objects of different kinds: for the ground variants it counts directives actually materialised by gringo, a faithful one-line-one-object count, whereas for the lazy variants it counts template facts that expand at run time into one candidate per ground target. The line count therefore *understates* the run-time work of the lazy backends by design; that work is measured separately by the per-decision metrics. All three caveats are taken up again in the discussion of the results.

3 Results

3.1 Functional Validation

The system was validated by keeping three levels separate. The first level concerns the ASP program itself: the answer sets, the `SAT`/`UNSAT` result, and the visible solutions must remain consistent across the compared encodings and across the two backends. The second level concerns the heuristic translation: targets, bodies, weights, priorities, modifiers, and aggregate bindings must be preserved when a native `#heuristic` directive is rewritten as a lazy fact, including the weight–priority flattening, which the suite applies uniformly to every variant and to both backends. The third level concerns the effect on search, measured through choices, conflicts, solving time, memory, and the size of the ground heuristic component.

Heuristics do not change the semantics of the ASP program: they guide search, but they do not add or remove answer sets. The equivalence harness of Section 2.6 verifies this property exhaustively on BSP, by enumerating all models of every variant on both backends and comparing the collections after projection on `b/1` and `c/1`, and by SAT/UNSAT agreement on PUP, where exhaustive enumeration is intractable. The same harness enforces the anti-fallback guards for the Prolog backend (`decide_calls > 0` for every lazy Prolog run, and equal choice counts between native and Prolog `la` on the control instance). On the native side, the C++ test suite isolates the correctness of the heuristic machinery from the benchmarks: it covers incremental aggregate states, empty and unsatisfiable corner cases, explicit body and target bindings, filtered aggregates, the local-versus-global interpretation of priorities under the two semantics, and the parser's rejection of malformed syntax.

3.2 Experimental Setup and Dataset

The campaign reported in this section was executed on an HPC cluster under SLURM, with one job per (system, setting, instance) triple orchestrated by `benchmark-tool`. Every run was limited by `runlim` to 600 seconds of real time and 30 GiB of resident memory, on 4 dedicated cores. Both backends were compiled on the compute nodes against the same toolchain, and the Prolog backend embeds a modern SWI-Prolog (10.0.2) built in user space for the same environment. The dataset covers the three families with their canonical instance ranges: BSP with $n = 10$ to 200 in steps of 10; PUP with the `double-$N$` instances, $N = 20$ to 200 in steps of 20 communication units; HRP with the reconfiguration instances `house-$N$`, $N = 2$ to 20 persons in steps of 2. All five shared settings run on all families; the three methodological extras (`ga_weak`, `la_aux`, `la_co`) are BSP-specific. The grand total is 520 runs (320 for BSP, 100 each for PUP and HRP), none of which ended in error: every unsolved run is an explicit timeout (T) or memout (M). Under this raised budget memouts are rare and concentrated: 11 runs out of 520, all on BSP, ten of them at exactly $n = 180$ (five variants on each backend, where the base program itself reaches the ceiling) and one for `la_aux` at $n = 170$; in every one of those runs the time limit was hit as well, so the two flags are raised together and the memout adds no separation. The dominant failure mode throughout the campaign is therefore timeout, and the frontiers reported below should be read as

time-bound rather than memory-bound except where stated.

The figures of this chapter follow the same separation. Every comparative plot shows the reference encoding and the four variants under study, `gc_noheur`, `gc`, `ga`, `la`, and `lc`; the three exploratory settings are kept out of these plots on purpose, because each of them answers a different, narrower question, and appear only in the dedicated analyses of Sections 3.8, 3.9, and 3.10, in figures restricted to the variants that each comparison actually needs.

Three properties anchor the validity of the numbers, and one qualifies them. First, within each system the ground variants (`g*`) execute identical binaries on identical ground programs, and clasp's search is deterministic at fixed configuration: choice and conflict counts are therefore exact structural measures, not noisy samples, and they reproduce bit-for-bit the counts observed in the preliminary local campaign (for example, `gc` at $n = 100$ performs exactly 32,514 choices on both machines). Second, the anti-fallback guard confirmed `decide_calls > 0` for every solved lazy Prolog run, so no lazy measurement silently degenerated to the baseline. Third, the two backends agree on the search trajectory wherever the heuristic is total and its ranking admits no ties: on BSP the choice and conflict counts of native and Prolog coincide to the unit in all 100 solved cells, ground and lazy alike. Where ties do occur the agreement is close but not exact — on PUP and HRP the lazy variants differ by a few percent in choices (for instance 217 against 197 for `la` at PUP $N = 160$), because the deterministic tie-break of the two implementations, a literal ordering in C++ and a sort key in Prolog, resolves equal-rank targets differently. All ground cells, on every family, remain bit-identical across backends.

What the Times Can and Cannot Carry

The qualification concerns *time*, and it must be stated before any number below is read. Jobs are dispatched by SLURM to whichever node of the `kr,kr-big` partitions is free, and those nodes are not identical. The effect is directly measurable, because three BSP variants (`gc_noheur`, `la`, `lc`) share the base encoding verbatim and therefore ground the same program to the rule (at $n = 120$, 52,759,256 rules against 52,759,258, the two extra being the lazy facts themselves): their grounding times should coincide, and instead they spread by a median of 13% and by as much as 52% at $n = 140$ (229.0, 343.7 and 347.4 seconds for the same program) and 47% at $n = 160$. At small sizes the spread is irrelevant in absolute terms; from $n \approx 100$ upwards, where grounding costs hundreds of seconds, it is large enough to reorder same-encoding variants in a time plot and, at the very top of the range, to decide on which side of the 600-second limit a run falls. Consequently: structural measures (choices, conflicts, rules, variables, atoms, ground objects, decide calls, candidates) are exact and are the basis of every claim in this chapter; time measures are reported as observed and are load-bearing only where the gap between variants is far larger than this spread, as it is for `ga` against `la` on PUP (a factor of two) or for `lc` against `la` on HRP (two orders of magnitude), and not where it is comparable to it, as it is between same-encoding variants at the top of the BSP range. Peak RSS is unaffected: it depends on the program, not on the node, and the same three variants agree to within 0.1% at every size.

Tables 2–4 summarize the outcome of every cell of the experimental matrix. The

columns report how many instances were solved and where the first failure occurs, distinguishing timeouts from memouts.

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	15/20	T@160	15/20	T@160
`gc`	14/20	T@150	14/20	T@140 (non-mon.)
`ga`	6/20	T@70	6/20	T@70
`ga_weak`	14/20	T@150	15/20	T@150 (non-mon.)
`la`	15/20	T@160	15/20	T@160
`lc`	16/20	T@170	14/20	T@150
`la_aux`	6/20	T@70	3/20	T@40
`la_co`	20/20	—	20/20	—

Table 2: BSP campaign overview ($n = 10..200$, 600 s / 30 GiB per run). At $n = 180$ five of the eight variants fail with both flags set (T and M simultaneously) on both backends, the one size at which the base program's own growth exhausts the memory budget at essentially the same point at which it would time out; `la_aux` reaches that point one size earlier. Two Prolog cells are non-monotone (`gc` fails at 140 but solves 150; `ga_weak` fails at 150 but solves 160). Above $n \approx 140$ all same-encoding variants are within the node-to-node spread of the grounding time discussed in Section 3.2, so the exact position of their frontiers should not be over-read: the separations that survive that spread are `ga` and `la_aux` at one end and `la_co` at the other.

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	4/10	T@100	4/10	T@100
`gc`	4/10	T@80 (non-mon.)	3/10	T@80
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	4/10	T@80 (non-mon.)	2/10	T@60

Table 3: PUP campaign overview (`double-`N, $N = 20..200$, 600 s / 30 GiB per run). Under this budget neither `ga` nor `la` fails anywhere in the tested range; the two variants whose heuristic carries no dynamic aggregate (`gc_noheur`, `gc`) stall by timeout well inside it. Two native cells are not monotone in N: both `gc` and `lc` fail on `double-80` but solve `double-100`. This is a property of the family — one instance per size, with neighbouring sizes of very different hardness — and Section 3.11 reads it through the choice counts rather than the times.

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	10/10	—	10/10	—
`gc`	10/10	—	10/10	—
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	8/10	T@18	7/10	T@16

Table 4: HRP campaign overview (`house-`N, $N = 2..20$ persons, 600 s / 30 GiB per run).

Three headline facts are already visible at this granularity and are unpacked in the following subsections. The first is that the three families put the frontier in three different places, and only two of them are informative. On PUP the frontier separates the variants by more than a factor of two in reach and is set by the heuristic's own grounding: it is the family where the comparison is decided. On HRP no variant except `lc` is ever in difficulty, so the frontier says nothing and the guidance quality says everything. On BSP the frontier of every same-encoding variant is set by the grounding of the base program, which the heuristic representation does not touch: `gc_noheur`, `la` and `lc` all stop between $n = 160$ and $n = 170$, within the node-to-node spread of Section 3.2, and the ordering among them is not a stable property of the campaign. What *is* stable on BSP, because it is structural, is that `la` reaches that common wall having performed exactly n choices and zero conflicts at every size, that is, with the search cost removed entirely rather than reduced.

The second fact is that two variants separate themselves from that band by a margin far larger than any measurement spread, and they do so in opposite directions. The ground Alpha simulation `ga` collapses on BSP precisely because of the static unrolling that makes it faithful, failing at $n = 70$, nine sizes before the heuristic-free baseline; and `la_co` solves the entire range in milliseconds. Third, the clingo-like lazy variant `lc` exhibits two distinct failure modes, inertness on BSP and active misguidance on HRP, that the per-decision instrumentation makes directly measurable, and it is the only variant whose frontier differs appreciably between the two backends, precisely because it is the only one whose cost is dominated by the per-decision machinery.

3.3 Reduction of the Ground Heuristic Component

The central claim of the thesis concerns *where* memory is spent, and the direct evidence is the number of ground heuristic objects as a function of n. The mechanism is readable in the encoding itself. The standard directive

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

carries the aggregate value S *as a variable inside the weight*. Since the atoms `c(Y)` are choice atoms, the grounder cannot know the sum at grounding time and

must materialize one directive for every reachable pair (X, S). With $X \in \{1, \ldots, n\}$ and S ranging over the subset sums of $\{1, \ldots, n\}$, which is every integer from 0 to $n(n+1)/2$ because 1 belongs to the domain, the count is

$$2 \cdot n \cdot \left(\tfrac{n(n+1)}{2} + 1 \right) \;=\; \Theta(n^3),$$

where the factor 2 accounts for the two symmetric rules for `b` and `c`. The formula is verifiable exactly on the measured data: for $n = 10$ it yields $2 \cdot 10 \cdot 56 = 1{,}120$, which coincides with the measured `ground_heuristics` value for `gc`. The explicit unrolling of `ga` materializes the same (X, S) product by construction. The lazy variants remove this enumeration at the root: the aggregate is evaluated at solving time on the current partial assignment, and the materialized heuristics remain constant, two facts, independent of n.

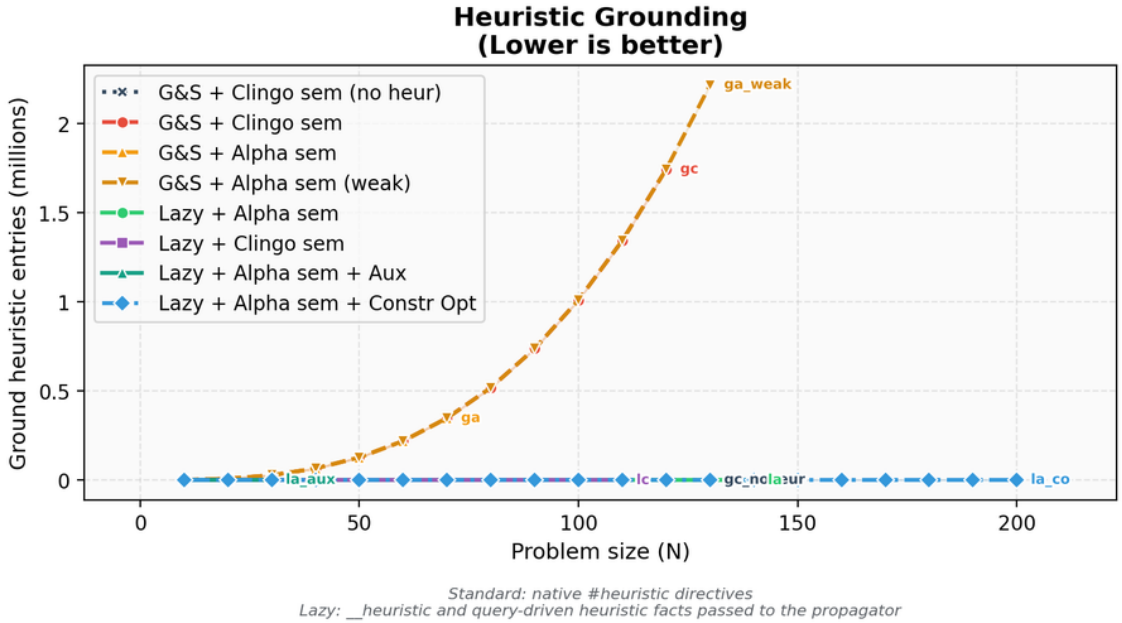

Figure 4: Comparison between native ground heuristic objects and lazy heuristic facts.

n	Native heuristic objects	Lazy facts	Ratio
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 5: Growth of the native heuristic component compared with the lazy representation. The counts are properties of encoding and instance, independent of the machine.

This comparison must be read for what it is. For the ground variants the count is faithful: one line is one materialized object. For the lazy variants the two counted

facts are *templates* that expand at runtime into one candidate per ground target (n candidates per rule for BSP); those candidates never pass through the grounder and therefore appear in no line count. The single-figure comparison "2 facts versus $\Theta(n^3)$ directives" is thus the proof of a saving in *grounding space*, not of a saving in total work: the work moved to runtime is real and is measured separately in Section 3.7.

3.4 BSP: Runtime Behaviour

Figure 5 reports the total time as measured by clingo, and Figure 6 its solving component. The base program dominates the picture for every same-encoding variant: at $n = 120$ the domain encoding grounds into 52.8 million rules, a component that `gc_noheur`, `la` and `lc` pay identically, since they share the encoding verbatim except for the heuristic representation (measured at 164.2, 163.9 and 117.4 seconds respectively, the difference being the node spread of Section 3.2 on a quantity that is provably the same). The total-time plot therefore has a wide band at the top of the range in which the same-encoding curves cross each other without meaning; what separates the variants, and what the next figure isolates, is the solving component and, for the ground heuristics, the additional grounding of the directives themselves.

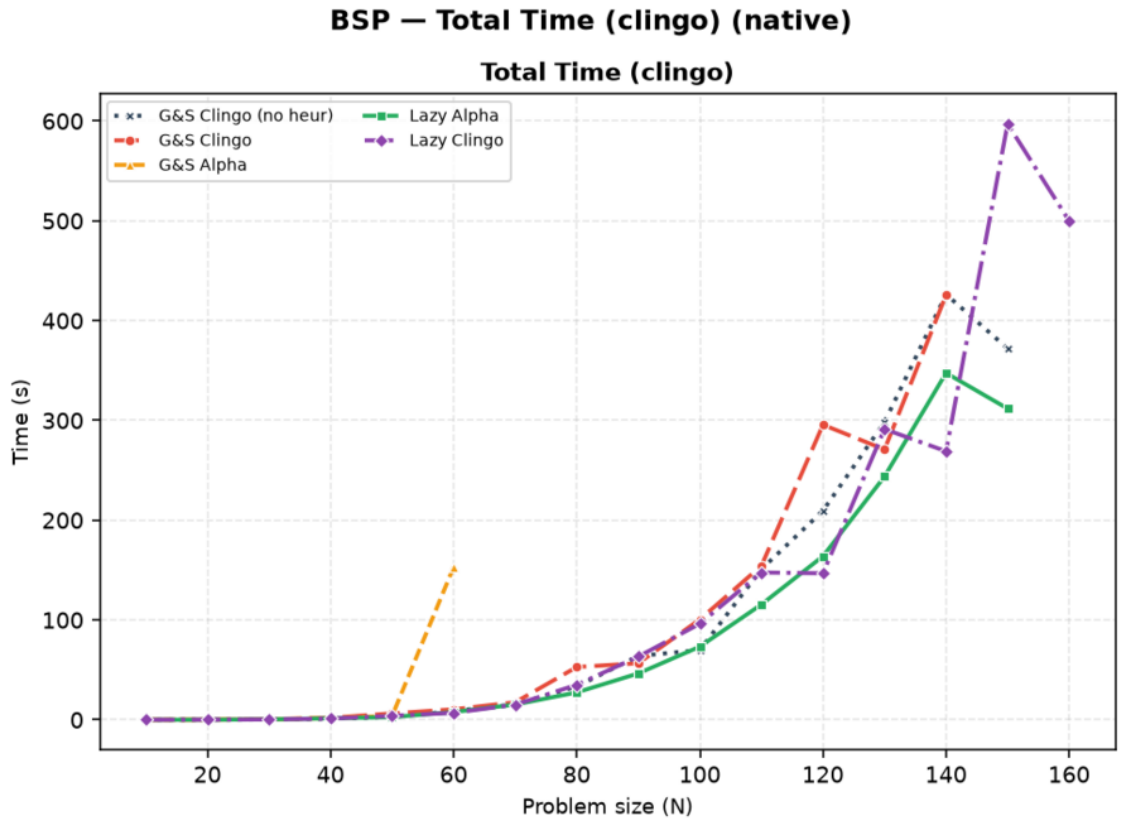

Figure 5: BSP, native backend: total clingo time. One run per point; a curve ends where its variant stops solving within the limits. Above $n \approx 120$ the three same-encoding curves (`gc_noheur`, `la`, `lc`) cross each other repeatedly: they ground identical programs, and the crossings are the node-to-node spread of Section 3.2, not a difference between variants. Figure 6 isolates the component that does separate them.

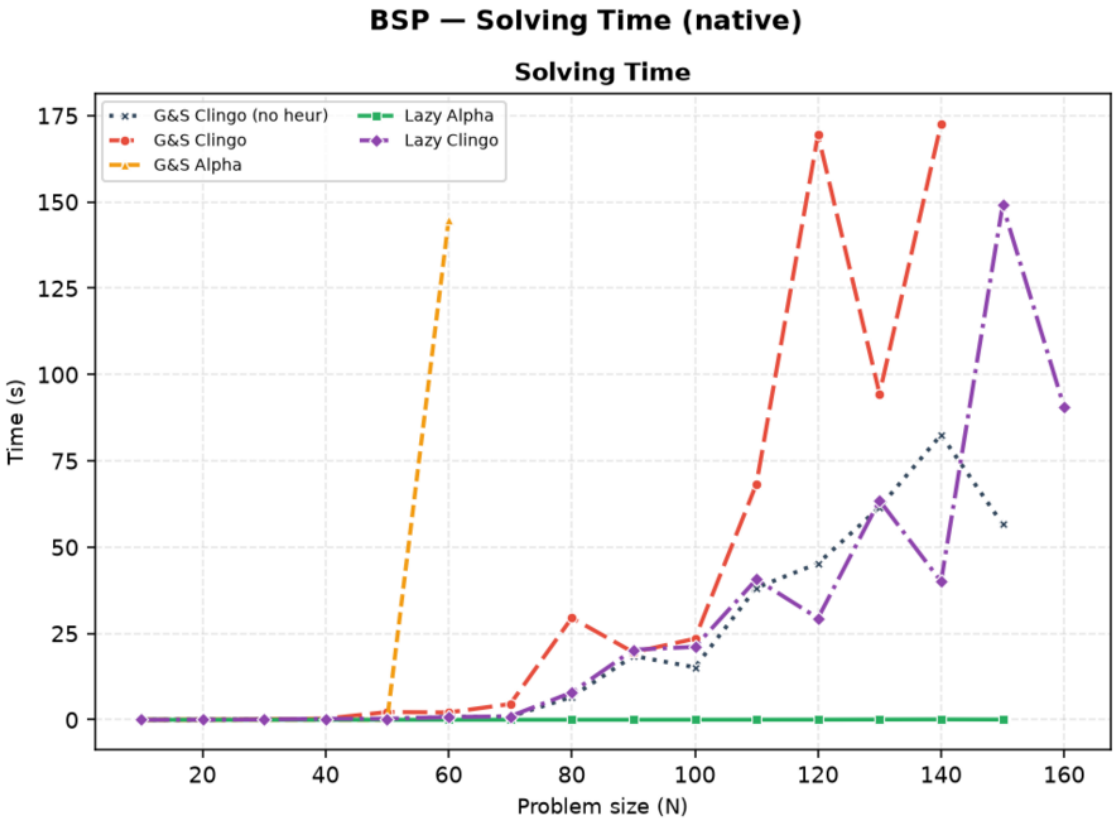

Figure 6: BSP, native backend: solving time only. `la` is indistinguishable from zero at every size; `ga` leaves the chart after $n = 60$.

Variant	Total (s)	Grounding est. (s)	Solving (s)	Choices	Conflicts
`gc_noheur`	209.5	164.2	45.3	78,565	49,993
`gc`	295.6	126.2	169.5	231,260	160,115
`ga`	T	(169.1)	(>430.9)	731,884	534,552
`ga_weak`	224.1	158.9	65.2	121,159	80,847
`la`	163.9	163.9	0.04	120	0
`lc`	146.8	117.4	29.4	78,565	49,993
`la_co`	0.007	0.007	0.00	117	0

Table 6: BSP at $n = 120$, native backend, under the 600 s / 30 GiB limits. T marks a timeout; the parenthesized solving value for `ga` is a lower bound (total minus grounding at the moment it was killed), and its choices/conflicts are the counts reached before the kill. `gc_noheur`, `la` and `lc` ground programs of the same size (about 52.76 million rules), and `gc`, `ga` and `ga_weak` programs of another common size (about 54.5 million): differences in the grounding column within each group are node spread, not variant cost, and only the solving column separates same-encoding variants reliably. Choices and conflicts are exact. `la_aux` is discussed in Section 3.9.

Three observations follow from Table 6. First, `la` turns search into a formality, 120 choices and zero conflicts, so its total time *is* the grounding time of the base

program, to four decimal places (163.902 against 163.862). This is the strongest form the claim can take on this family, and it is stronger than a comparison of totals: the solving column, which is the only one the heuristic controls, drops from 45.3 seconds on the baseline to 0.04, a factor of a thousand, while every other cost in the run is left exactly where the encoding put it. Under the raised time budget `gc` now finishes at $n = 120$ as well, but pays 169.5 seconds of solving where `la` pays 0.04, and it is `gc`, not `la`, that first crosses into timeout territory as n grows further (Table 2: `gc` fails from $n = 150$, `la` from $n = 160$, at which point the base program alone consumes the entire budget).

Second, `ga` is the surprise of the full campaign, and a negative one. Its static unrolling was designed as the faithful ground simulation of the Alpha reading, and Table 5 shows it pays the same $\Theta(n^3)$ grounding as `gc`; but unlike `gc`, whose negative guards deactivate directives as atoms become established, the unrolled directives of `ga` lack those guards by design, so the Domain heuristic is flooded with simultaneously applicable modifications. The search degenerates: at $n = 60$, `ga` spends 144.9 seconds of solving against 0.85 of the baseline, with 1,668,059 choices against 11,810, and from $n = 70$ it never finishes within the limit even at 600 seconds (at $n = 120$ it was killed at 731,884 choices and counting). This is a gap of more than two orders of magnitude on a structural measure, so it is entirely unaffected by the caveat of Section 3.2. The faithful simulation is thus *doubly* punished, once in the grounder and once in the solver, so severely that it is the only BSP heuristic variant, ground or lazy, that fails before the heuristic-free baseline does, and by nine sizes: an instructive datum for anyone hoping to emulate dynamic heuristics in a ground-and-solve pipeline.

Third, Table 6 contains two rows that the figures of this section deliberately omit. `ga_weak`, the cheap approximation of the Alpha reading, stays close to the baseline and is compared against the faithful `ga` in Section 3.8; `la_co`, which changes the problem model itself, is the subject of Section 3.10.

3.5 BSP: the Two Semantics Under the Microscope

Search statistics separate the two semantics sharply, and the full campaign adds an instrumentation-level proof of *why*. The `la` variant makes exactly n choices with zero conflicts at every size: the Alpha reading makes the greedy balance heuristic total, one direct decision per element, which is the known optimal strategy for this problem. The `lc` variant is not merely "close to the baseline": it is *search-identical* to it. At every size, its choice and conflict counts coincide with `gc_noheur` to the last unit (78,565 and 49,993 at $n = 120$).

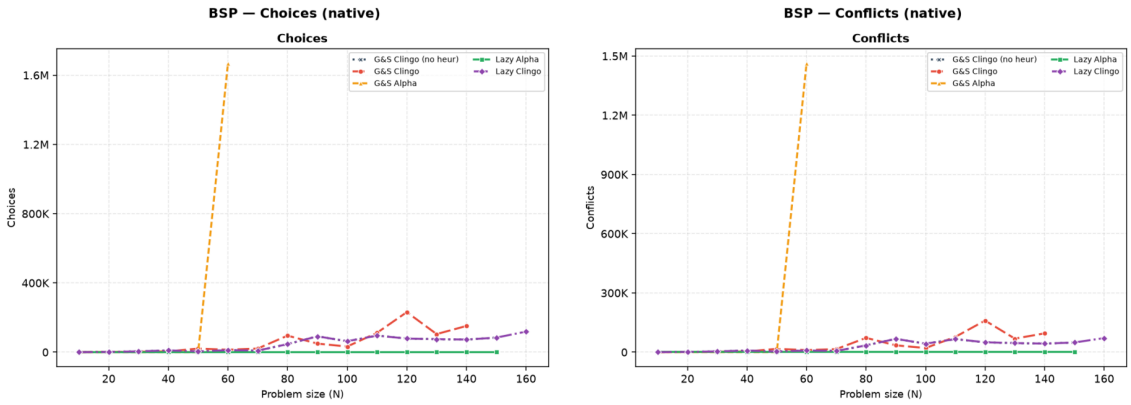

Figure 7: BSP, native backend: choices and conflicts. `la` lies on the horizontal axis; `lc` coincides with the baseline exactly, at every size both solve, so the two curves are superimposed; `ga` leaves the chart after $n = 60$. Unlike the time plots, these are exact structural counts.

The instrumentation explains the coincidence, and both backends now report it independently. On BSP the `lc` rule guards the aggregate source with the clingo reading: the sum over `c/1` becomes usable only when every `c` atom is determined, and `clingo_not(c(X))` additionally requires established falsity. During search these conditions are essentially never satisfied at decision time: at $n = 120$ both backends record exactly 78,565 decide calls — one per solver choice, the propagator is consulted every single time — with an average of *zero* applicable candidates per call (`avg_candidates_per_decide` = 0.0, and `max_candidates_seen` = 0, so not a single call in the whole run produced a candidate). Every decide call returns nothing, the solver falls back to its default heuristic, and the trajectory reproduces the baseline exactly. The clingo-like lazy variant on BSP is therefore *inert by semantics*: it costs the full decide-loop machinery and buys nothing, which is precisely what the design predicts for a heuristic whose guards wait for information that only exists at the bottom of the search tree.

3.6 BSP: Rules, Variables, and Memory

The compression of the heuristic component is visible in the size of the propositional instance. At $n = 100$, `gc` is fifty times larger in solver variables than every other variant, because its heuristic directives materialize the negative body and the aggregate bound for each (X, S) pair. Notably, `ga` does *not* inflate the variable count (20,402, in line with the baseline): its unrolled directives reuse the same aggregate structures, and its cost shows up as grounding time and unguarded activations instead. The two ground variants thus pay the same combinatorial product in two different currencies.

Variant	Variables at $n = 100$
`gc`	1,030,606
`ga`	20,402
`ga_weak`	20,406
`lc`	20,300
`la`	20,300
`gc_noheur`	20,300

Table 7: Number of solver variables at $n = 100$, native backend.

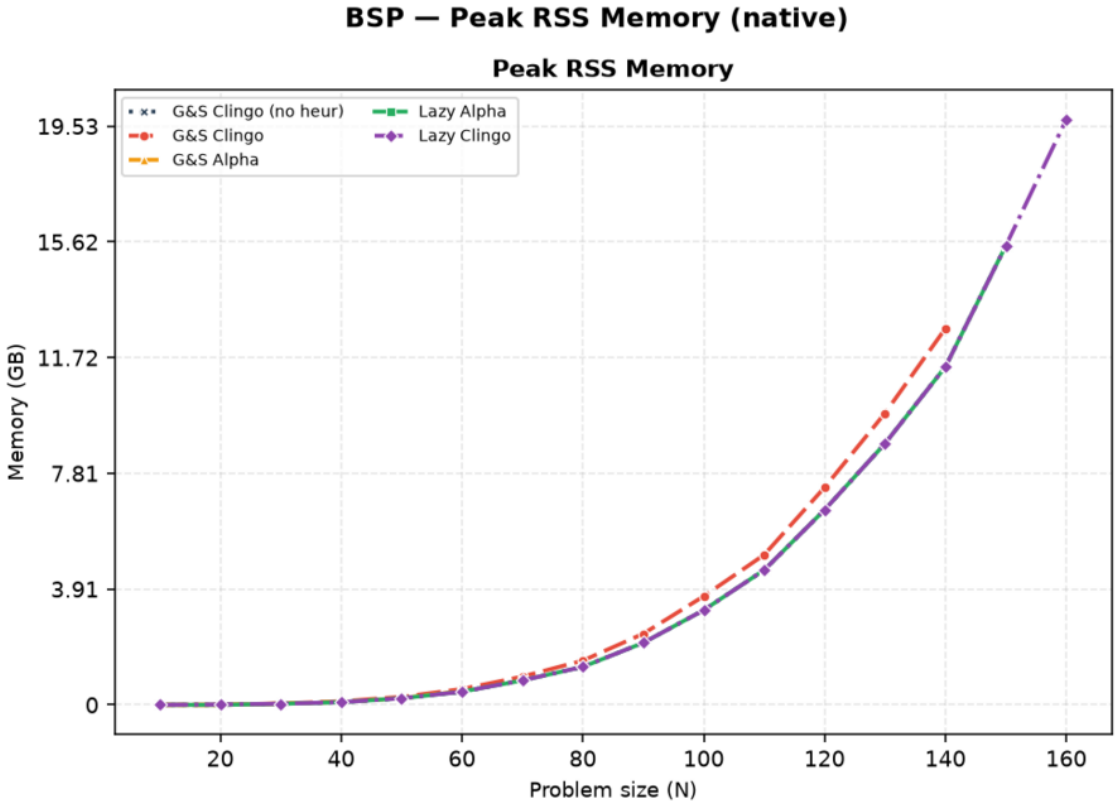

Figure 8: BSP, native backend: peak resident set size.

The memory metric is the peak resident set size of the entire solver process, end to end: it covers grounding, CDCL search, and the custom propagator in a single value, and for the lazy Prolog variants it includes the in-process SWI-Prolog engine and its asserted database. Unlike the time measures, it is stable across nodes: the three same-encoding variants agree to within 0.1% at every size, which is itself a useful control on the metric. At large n memory is dominated by the base program for every same-encoding variant: at $n = 120$, `gc_noheur`, `la`, and `lc` sit at 6.74, 6.74 and 6.74 GB, while `gc` reaches 7.5 GB and `la_aux` 8.7 GB. Under the 30 GiB budget none of these is close to the ceiling at this size; the base program only approaches it much further out, at $n = 180$, which is where Table 2 records the campaign's only memouts. The heuristic saving is real but bounded by the base program's own

footprint; the variant that changes the footprint by orders of magnitude is `la_co` (Section 3.10), because it changes the base program itself. On the Prolog side the SWI engine adds a fixed overhead of the order of ten megabytes, invisible at this scale and dominant only on the trivial `la_co` runs. The RSS *confirms* the saving on the real consumption; the ground-object count of Table 5 *explains its cause*.

3.7 The Price of Laziness: Per-Decision Cost

Intellectual honesty requires stating that the lazy approach does not *eliminate* the cost of the combinatorial explosion: it *moves* it. What ground-and-solve pays once, in memory, at grounding time, the lazy backend pays repeatedly, in time, at every decision. The full campaign turns this from a declared principle into measured numbers, and both backends are now phase-timed, so the cost can be attributed rather than inferred: the propagator reports, per run, its decide calls, the candidates each call produced, the cumulative time spent answering and the cumulative time spent keeping its state synchronized with the trail.

				ms/decide			
Family	**Variant**	**decide calls**	**cand./decide**	**native**	**Prolog**	**sync (s)**	**prop. share**
BSP n=120	`la`	120	121	$1.1{\cdot}10^{-4}$	0.58	0.003	0.0%
BSP n=120	`lc`	78,565	0.0	$0.8{\cdot}10^{-4}$	0.21	3.34	8.7%
PUP N=160	`la`	197	5.3	$2.1{\cdot}10^{-4}$	2.54	5.74	6.1%
PUP N=200	`la`	249	5.2	$2.5{\cdot}10^{-4}$	6.60	16.13	7.2%
HRP N=20	`la`	106	1,248	$1.1{\cdot}10^{-4}$	9.52	0.035	64.9%
HRP N=14	`lc`	73,143	239	$0.8{\cdot}10^{-4}$	3.46	11.15	99.4%

Table 8: Propagator instrumentation on representative solved runs. *decide calls* and *cand./decide* are taken from the Prolog backend, which counts every candidate the `heuristic/4` query returns; *ms/decide* is the average time of one consultation, reported by both backends; *sync* is the cumulative cost of mirroring the solver trail into the Prolog database; *prop. share* is the fraction of the Prolog run spent inside the propagator (query plus synchronization). The native backend's own synchronization is not shown because it never exceeds 0.3 seconds in the whole campaign.

Table 8 decomposes the overhead along its two multiplicative factors: how often the heuristic is consulted (decide calls, driven by the search trajectory) and how much each consultation costs (driven by the number of candidates the rules generate and by the size of the synchronized state). Before reading the corners, one figure dominates the table and deserves to be stated on its own: the per-consultation cost differs between the two backends by three to four orders of magnitude, from 10^{-4} milliseconds for the compiled C++ evaluation against 0.2 to 9.5 milliseconds for the SWI-Prolog one. This is the price of generality, measured, and it is the reason the two backends occupy different points of the design space rather than competing on the same one.

The four corners of the space are all realized in the data. `la` on BSP is the benign corner: 120 consultations of half a millisecond each, 73 milliseconds of propagator time on a 164-second run, so the Prolog and native totals coincide to within a tenth

of a second (163.98 against 163.90) — the clearest demonstration in the campaign that when the heuristic guides perfectly, even an entire embedded logic-programming engine is free. `lc` on BSP multiplies a *small* per-call cost by 78,565 inert consultations; the query cost stays low (a fifth of a millisecond each), but the count is large enough that the propagator ends up owning 8.7% of the run — 14.7 seconds of queries and 3.3 of synchronization — for a heuristic that never contributes a single decision. `la` on HRP multiplies *few* consultations by a large per-call cost, since the HRP reference rules [7] enumerate more than a thousand candidates per decision; on sub-second instances the 1.05 seconds of query time it accumulates *are* the run (64.9% of it), turning a 0.29-second native run into a 1.61-second Prolog one (×5.5). `lc` on HRP combines both factors, 73,143 consultations at 3.5 milliseconds each: 240 of its 266 seconds go into the queries and 11 more into synchronization, 99.4% of the run, against 1.36 seconds for the native backend on the same instance — the largest cross-backend gap in the campaign (×196). Synchronization, finally, scales with the trail activity and the number of watched atoms and is the one component that grows with instance size rather than with search: on PUP `la` it dominates the query cost by more than an order of magnitude (5.74 against 0.50 seconds at N=160, 16.13 against 1.63 at N=200), which is what makes the batched-update protocol of the future work a quantified target rather than a guess.

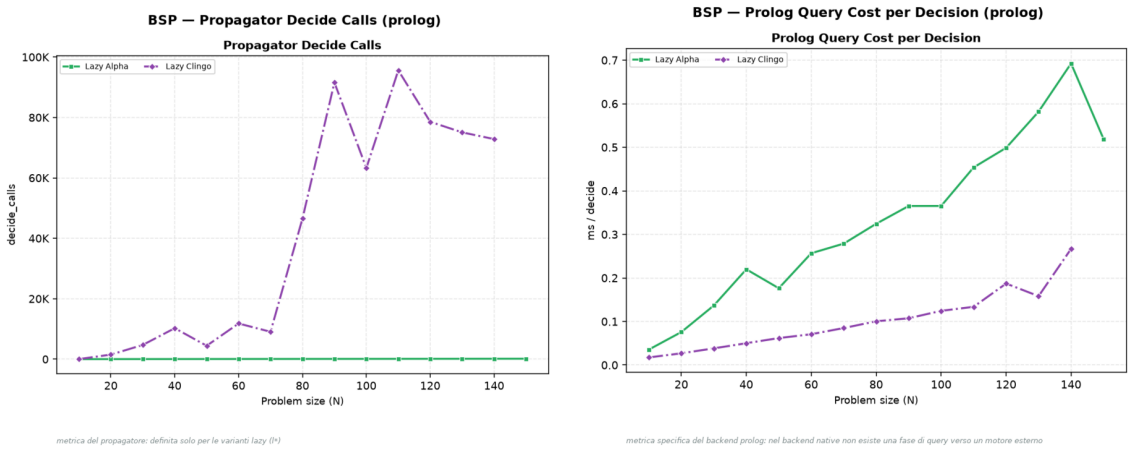

Figure 9: BSP, Prolog backend: decide calls and per-decision query cost. The `lc` curve pairs tens of thousands of calls with a low unit cost; `la` the opposite.

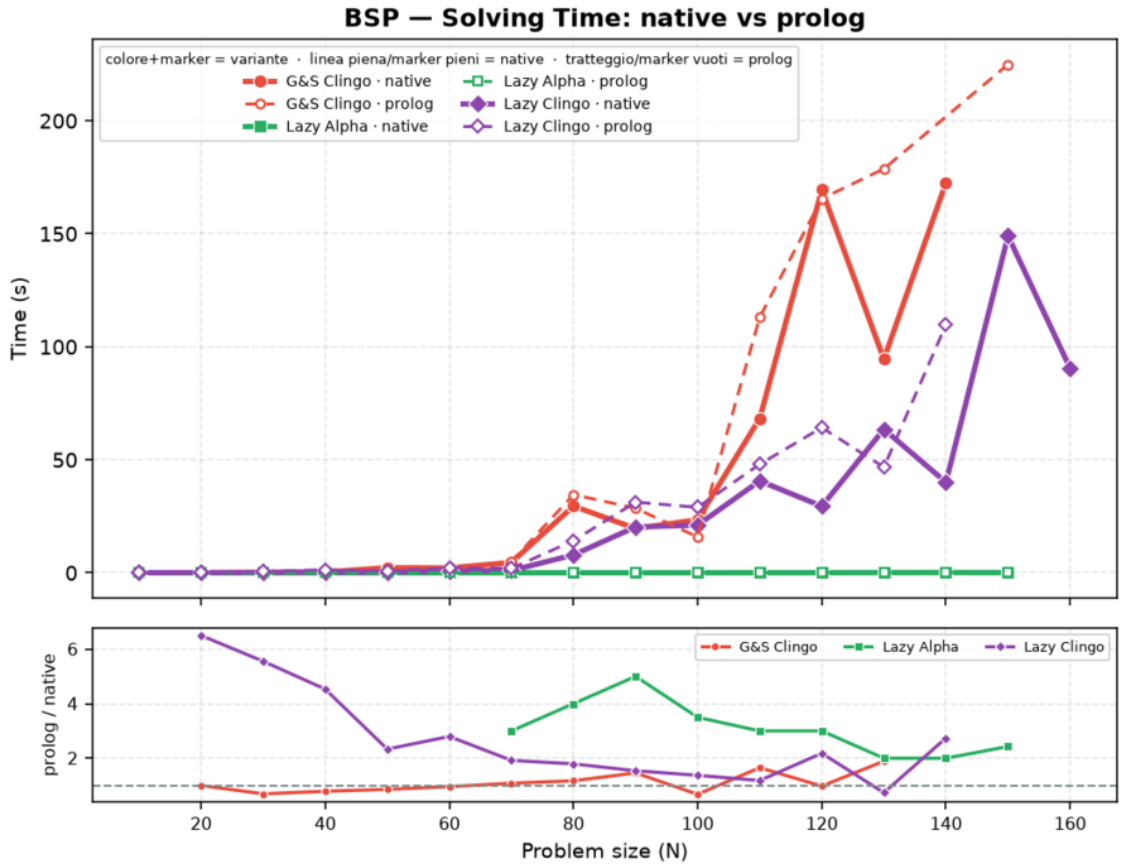

Figure 10: BSP: solving time, native (solid) versus Prolog (dashed) backend, lazy variants and ground reference.

The general law suggested by the table, and consistent with all 520 runs, is that the lazy overhead is the product *decide calls* × (*query cost* + *amortized sync*), where the first factor is governed by how well the heuristic guides (a good heuristic consults itself n times; a bad or inert one, once per baseline choice) and the second by how much of the program the rules can see. The practical reading is that *lazy heuristics are cheapest exactly when they work*: effective guidance keeps the decision count, and therefore the total overhead, small.

3.8 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

The three subsections that follow analyze the exploratory settings. They are not alternative candidates in the main comparison: each one isolates a single design decision, and its figures include only the variants that decision touches.

The first decision is how faithfully the Alpha reading can be forced into a ground-and-solve pipeline. `ga` is the faithful simulation and pays for both halves of it: it drops the negative literals so that established truth does not deactivate the directives, *and* it statically unrolls the aggregate, replacing the equality binding of S with the monotone bound $S \leq \#\texttt{sum}\{Y : c(Y)\}$ over every candidate value, so that clasp's max-weight selection reconstructs the current value of the growing sum at solving time (Section 2.1). `ga_weak` keeps only the first half: it treats the negation as

Alpha does and leaves the aggregate untouched, which means the grounder evaluates the sum once, on the empty assignment, freezing $S = 0$ and degrading the heuristic to a static ranking of the elements by X.

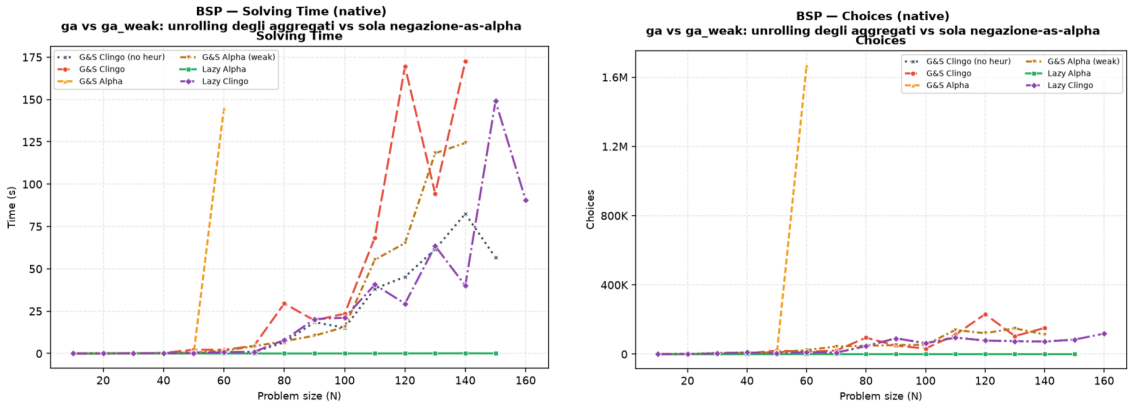

Figure 11: BSP, native backend, exploratory comparison: solving time (left) and solver choices (right) with both Alpha simulations alongside the main variants. `ga` degenerates at $n = 60$ (1.7 million choices, 145 seconds of solving) and times out from $n = 70$; `ga_weak` tracks the baseline over the whole range.

Figure 11 shows what each half costs. The unrolled `ga` explodes exactly where the unguarded activations flood the Domain heuristic: at $n = 60$ it needs 1,668,059 choices and 144.9 seconds of solving against 11,810 choices and 0.85 seconds of solving (8.8 in total) of the baseline, and from $n = 70$ it never finishes even at the raised 600-second limit, the double punishment already diagnosed in Section 3.4. `ga_weak` sits at the opposite end: structurally it is indistinguishable from the baseline (same rules, constraints, and variables, 20,406 versus 20,300 at $n = 100$, because nothing is unrolled), and behaviourally it stays close to it over most of the range (65.2 versus 45.3 seconds of solving at $n = 120$, with somewhat noisier search: 121,159 choices against 78,565). It solves fourteen of the twenty instances, one size short of the baseline itself (timeout from $n = 150$ against $n = 160$), precisely because it renounces the dynamic part of the heuristic; the frozen $S = 0$ ranking neither guides nor obstructs, so it tracks the baseline's own cost closely without ever improving on it. The contrast with `la` is the cleanest available on this family, because all three encodings ground the same order of magnitude of rules and differ only in what happens during search: 0.04 seconds and 120 choices for the dynamic aggregate evaluated lazily, 65.2 seconds and 121,159 choices for the same aggregate frozen at grounding time, 430 seconds and counting for the same aggregate unrolled.

Read together, the two simulations bracket the lazy propagator from both sides and confirm that the benefit measured for `la` comes from the *dynamic* aggregate, not from the mere presence of weights or from the Alpha activation rule alone. The half of the Alpha reading that is cheap to ground (`ga_weak`) is behaviourally inert, and the half that reproduces the guidance (`ga`) is combinatorially unaffordable; only the runtime evaluation of the aggregate obtains the guidance without the unrolling, which is the design point the lazy propagator occupies.

3.9 Direct and Auxiliary Forms

The `la_aux` variant routes the lazy heuristic through auxiliary predicates that materialize the heuristic body, including the aggregate value, before the propagator is used. This experiment separates two effects that are otherwise easy to confuse: reducing the number of ground heuristic directives, and avoiding the materialization of the heuristic body itself. `la_aux` keeps only two `__heuristic` facts, but its auxiliaries `h_b(X,S)` and `h_c(X,S)` reintroduce the (X, S) product in the base program, and the full campaign shows the consequences from three angles at once. In the grounder, the auxiliary atoms inflate the instance (145,600 solver atoms and 132,756 variables at $n = 50$, against 10,352 and 5,150 for `la`). In the propagator, every (X, S) atom becomes a runtime candidate, so the per-decision scan grows with n^2: the native backend already spends 278.8 seconds of *solving* to finish $n = 60$ (against 0.00 for `la`), and from $n = 70$ it times out at the raised 600-second limit; its peak RSS keeps climbing with n regardless (8.7 GB already at $n = 120$, on its way to a memout at $n = 170$ where the auxiliary atom count alone exceeds four million). On the Prolog side the same candidate flood must additionally cross the synchronization bridge, and there it becomes the whole run: at $n = 20$ it already needs 2.9 seconds of cumulative synchronization for 573 decide calls, against 0.3 milliseconds for the 20 calls of `la`, and at $n = 30$ synchronization alone accounts for 363 of the 366 seconds of the run (99.3%), with the queries themselves costing 23 milliseconds in total. The variant times out from $n = 40$. The signature is worth naming, because it is the opposite of the HRP one: there the propagator was expensive because each query was expensive; here each query is *cheap* (0.02 ms, the cheapest in the campaign) and the cost is entirely in keeping a state that the auxiliary relation made large. The lesson is clear: if the heuristic body is first materialized into a large auxiliary relation, the intended benefit of laziness is lost on every axis at once, and it is the search cost of the reintroduced product, not memory alone, that dominates the failure. The direct lazy form is the meaningful one.

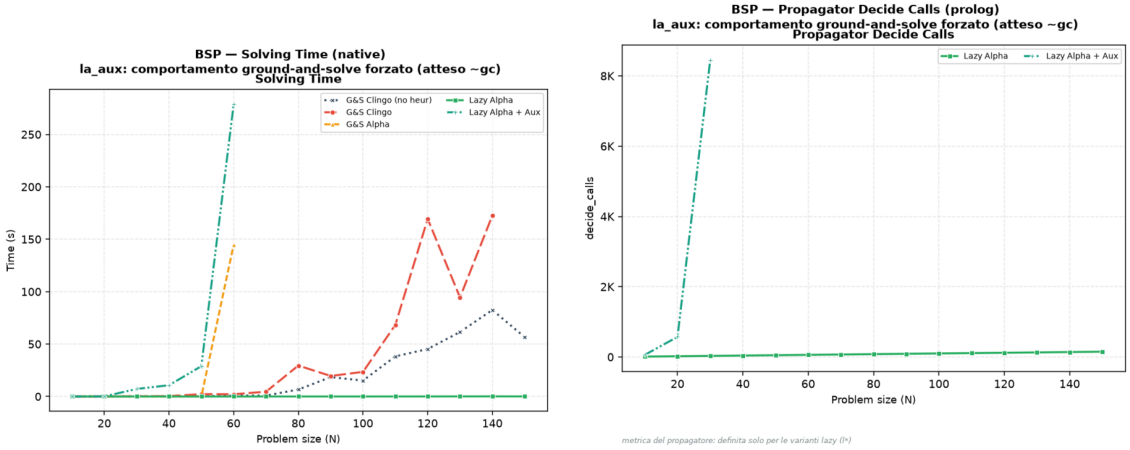

Figure 12: BSP, exploratory comparison of the auxiliary form. Left: native solving time; `la_aux` diverges from $n = 40$ and leaves the chart after $n = 60$, while the direct `la` stays on the axis. Right: propagator decide calls on the Prolog backend; the auxiliary candidates multiply the consultations (573 at $n = 20$ and 8,452 at $n = 30$, against the n calls of the direct form), and the curve ends at $n = 30$, the last size the variant solves on that backend.

3.10 Effect of the Constraint Formulation

The `la_co` variant changes not only the heuristic representation but also the BSP constraint itself, replacing the quadratic difference check over `sumB`/`sumC` with a single interval constraint over one sum, whose bounds are compile-time constants. The effect is dramatic and worth stating exactly: at $n = 120$ the propositional instance drops from 52,759,258 rules to 366 and from 29,160 variables to 121, and the clingo time from 209.5 seconds (baseline) or 163.9 seconds (`la`) to 7 milliseconds, with the same 117 guided choices and zero conflicts, on both backends (the Prolog run measures 98 milliseconds, the difference being the fixed engine startup). The same holds at every size up to $n = 200$, where `la_co` still finishes in 9 milliseconds with 197 choices while every other variant has been timing out for four sizes. This measurement completes the diagnosis of Section 3.4: once the lazy heuristic has removed the search cost, the *entire* residual cost of BSP is the ground materialization of the balance check, and a formulation that avoids it removes the last Θ-large term. A comparison between `la_co` and `la` therefore measures the interaction between lazy heuristics and problem modelling, not the isolated effect of the heuristic representation, and it is reported as a modelling experiment: its role is to show what the combination "dynamic heuristic + propagation-friendly model" can reach when neither component pays a combinatorial ground price.

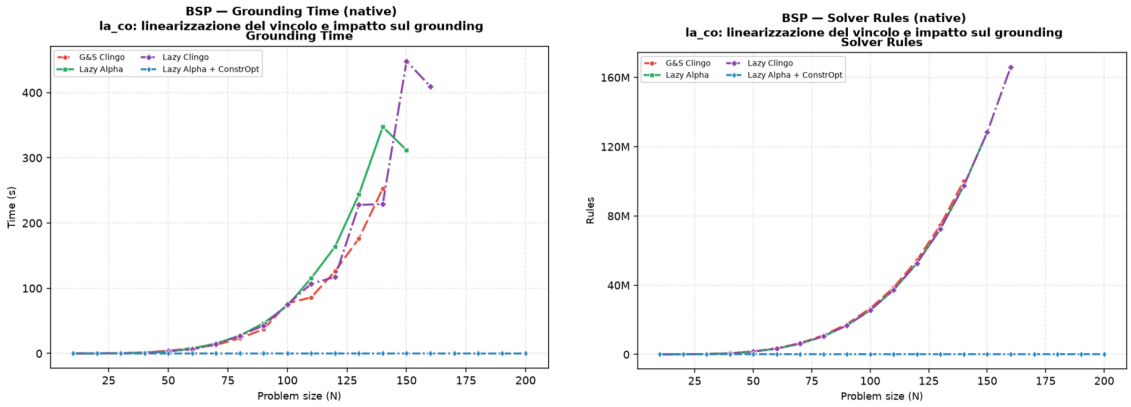

Figure 13: BSP, native backend, exploratory comparison of the constraint reformulation: grounding time (left) and propositional rules (right). `gc`, `la`, and `lc` overlap because they share the quadratic base encoding (some 52.8 million rules and one to three minutes of grounding at $n = 120$); `la_co` lies on the horizontal axis of both plots (366 rules, milliseconds of grounding), the visual form of the five-orders-of-magnitude gap discussed in the text.

3.11 PUP: the Grounding Bottleneck at Scale

PUP is the family where the grounding bottleneck, not the search, sets the frontier, and where the lazy representation translates directly into a large saving in time and memory at every common size. Figure 14 and Table 9 summarize the campaign under the 600-second, 30 GiB limits; under this raised budget both Alpha-like variants solve the entire tested range up to $N = 200$, so the frontier that used to separate them has become a cost gap instead.

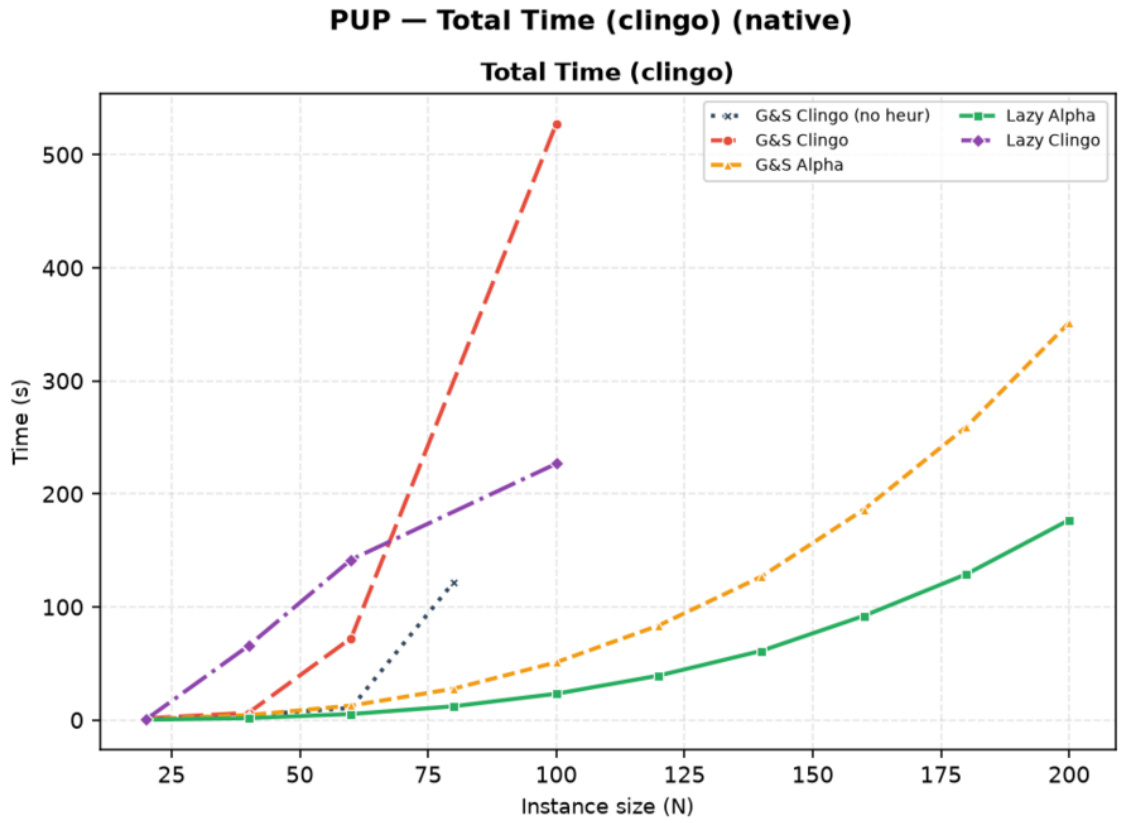

Figure 14: PUP, native backend: total clingo time. Each curve ends at the last instance its variant solves.

Variant	max N solved	Total (s)	Grounding (s)	Peak RSS	first failure
`gc_noheur`	80	122.0	10.3	0.7 GB	T@100
`gc`	100	527.3	61.3	4.0 GB	T@80 (non-mon.)
`ga`	200	351.4	346.5	25.0 GB	—
`la`	200	176.9	173.9	13.0 GB	—
`lc`	100	226.7	26.7	2.1 GB	T@80 (non-mon.)

Table 9: PUP frontier, native backend, 600 s / 30 GiB per run: largest `double-`N solved and its cost. `ga` and `la` solve the whole tested range and are reported at $N = 200$; the other three still fail inside the range, and are reported at the largest size they solve. Unlike the BSP table, the separations here are between programs of genuinely different size — `ga` grounds twice the memory of `la` at every size — and are therefore an order of magnitude larger than the node spread of Section 3.2.

The ordering of the frontiers tells most of the story, and the part that changes with scale tells the rest. The baseline stalls at $N = 80$ by *time*: with no guidance, search explodes first (546,549 choices at $N = 80$, against 272 for `la` at $N = 200$, two and a half times that size). The plain ground heuristic `gc` fares no better, failing already at $N = 80$: its directives are cheap to ground but do not carry the dynamic aggregate, so they neither prevent the search explosion (1,861,373 choices at $N = 80$, three times the baseline's) nor pay for themselves; that it then solves $N = 100$ is

the non-monotonicity of the family, not a trend. Both Alpha-like variants clear the whole tested range under the raised budget, but at markedly different cost. `ga` pays for its static unrolling on every instance: at $N = 120$ it needs 83.6 seconds and 7.0 GB against `la`'s 39.3 seconds and 3.4 GB, a 2.1× time and 2.0× memory gap that persists at every common size, because the unrolled directives grow with the value domains being unrolled, not just with the instance; at $N = 200$ it is 351.4 against 176.9 seconds (1.99×) and 25.0 against 13.0 GB (1.93×). Both variants show the same near-perfect search quality (235 choices and zero conflicts for `ga` against 162 choices and 16 conflicts for `la` at $N = 120$; both semantics-Alpha), so the entire gap is attributable to the heuristic's own grounding, exactly as the mechanism predicts. Two independent facts make this the most robust comparison in the campaign: the gap is a stable factor of two across ten sizes, an order of magnitude above the node spread of Section 3.2, and it shows up identically in peak RSS, which that spread does not affect at all. Extrapolating the trend, `ga` is the variant one would expect to hit a memory wall first if the range were pushed further, since it reaches 25.0 of the available 29.3 GiB already at $N = 200$ while `la` is at 13.0; the campaign as run does not observe that wall directly, but the ratio is the same signature that, under the previous 8 GiB budget, made `ga` the first of the two to fail.

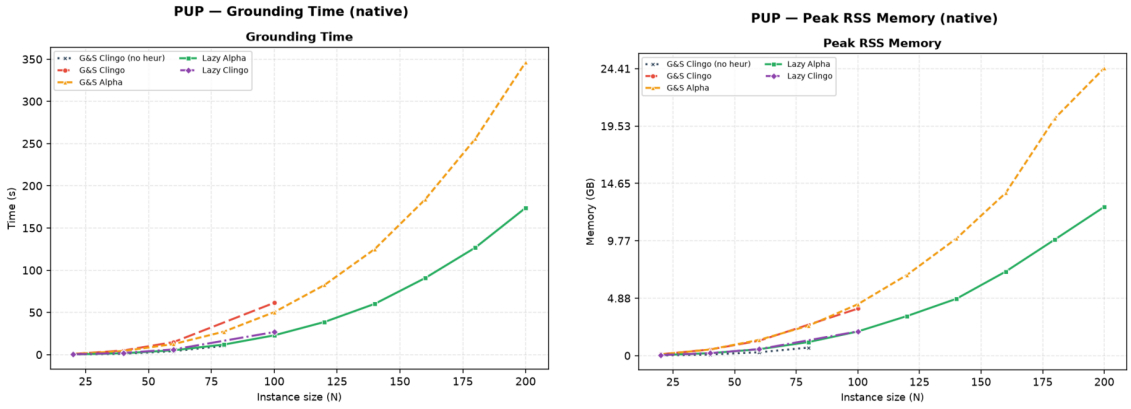

Figure 15: PUP, native backend: grounding time and peak memory. `ga` pays the heuristic unrolling (orange) on top of the base growth; `la` (green) pays the base program only, and stays consistently cheaper at every common size.

The clingo-semantics cells complete the picture from below. Native `lc` oscillates around the baseline, solving $N = 100$ (which the baseline misses) but failing at $N = 80$ (which the baseline solves): as on BSP, its gated aggregates never produce candidates (on every instance it solves, both backends report *zero* candidates and a `max_candidates_seen` of zero, over 233 and 36,325 native decide calls). Unlike BSP, however, the trajectory does diverge from the baseline — 36,325 choices against 11,659 at $N = 40$, 174,593 against 3,329,783 at $N = 100$ — which is at first sight a contradiction: a propagator that never returns a candidate cannot change a decision. The resolution is in the ground programs. On BSP the lazy encoding and the baseline instantiate the same program up to the two `__heuristic` facts (52,759,258 rules against 52,759,256), which is why an inert propagator there implies a bit-identical trajectory. On PUP the lazy encoding needs auxiliary predicates to expose

the aggregate sources, so it grounds a visibly different instance (216,193 rules and 51,776 variables at $N = 20$, against 161,018 and 5,298 for the baseline): clasp sees a different formula and, on a family this sensitive to it, takes a different trajectory from the first restart. The perturbation is therefore incidental rather than heuristic in origin, it moves in both directions, and it is the honest reading of the non-monotone frontier reported in Table 3.

What this same fact buys, however, is the cleanest semantics experiment of the campaign. On PUP `1a` and `1c` ground *exactly* the same program — identical rules, atoms, variables and constraints at all ten sizes — and differ only in the `__semantics` token that selects how the propagator reads partial information. Everything that separates them is therefore the semantics and nothing else: at $N = 120$, 162 choices and 39.3 seconds against 81,028 choices and 600 seconds; at $N = 200$, 272 choices against a timeout. The same comparison on the ground side is impossible to set up, because `ga` and `gc` necessarily differ in their unrolling as well. On the Prolog backend the same inert consultations acquire a price (25,197 calls and 80.1 seconds of synchronization, 76% of the run, already at $N = 40$), and `1c` collapses to $N = 40$: the pathological corner of Section 3.7 reproduced at scale.

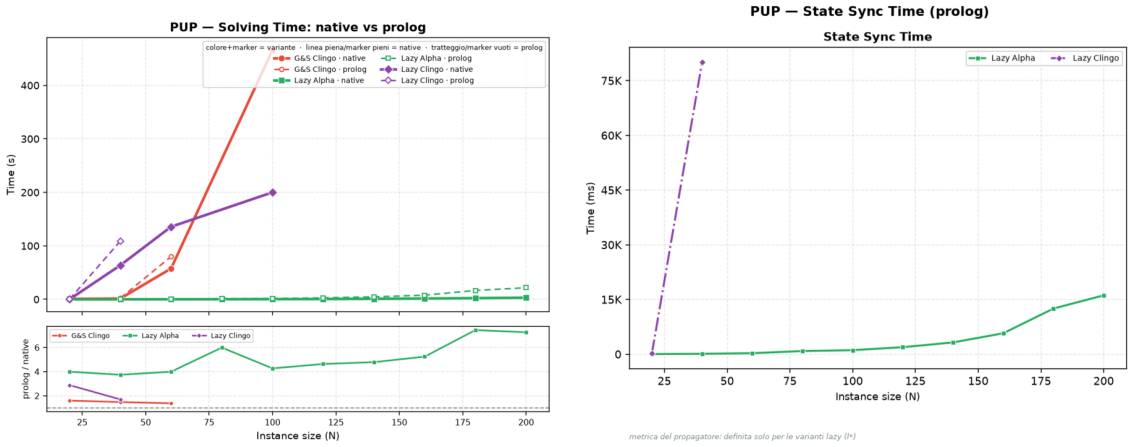

Figure 16: PUP: native-versus-Prolog solving comparison (left) and cumulative synchronization cost of the Prolog backend (right). For `1a` the synchronization, not the query, is the dominant lazy cost at scale.

For the well-behaved `1a`, the cross-backend comparison isolates the pure cost of the general-purpose engine: the guidance is equivalent (both backends solve the whole range, with search statistics within a few percent of each other — 217 against 197 choices at $N = 160$ — the residual difference being the tie-break discussed at the start of this chapter), and the Prolog total grows from 92.1 to 102.0 seconds at $N = 160$ (+11%) and from 176.9 to 245.3 at $N = 200$ (+39%), of which 5.7 and 16.1 seconds respectively are trail synchronization, against 0.5 and 1.6 of query time. At PUP scale the bottleneck of the embedded engine is thus *keeping the state*, not answering the queries — it costs eleven times the queries it serves, and its share of the run grows with N — which points directly at the batched-synchronization optimization discussed in future work. The native backend, whose synchronization is incremental and in-process, spends 0.07 seconds on the same task at $N = 200$.

3.12 HRP: Semantics Without Aggregates

HRP is the negation-heavy benchmark: its five-level reference heuristic [7] contains no aggregates, so the `la`/`lc` comparison isolates the negation semantics alone. At the benchmark sizes the problem is easy for clasp, every variant except `lc` solves every instance in well under a second on the native backend, which makes HRP useless for frontier claims but ideal as a controlled experiment on guidance quality and per-decision cost.

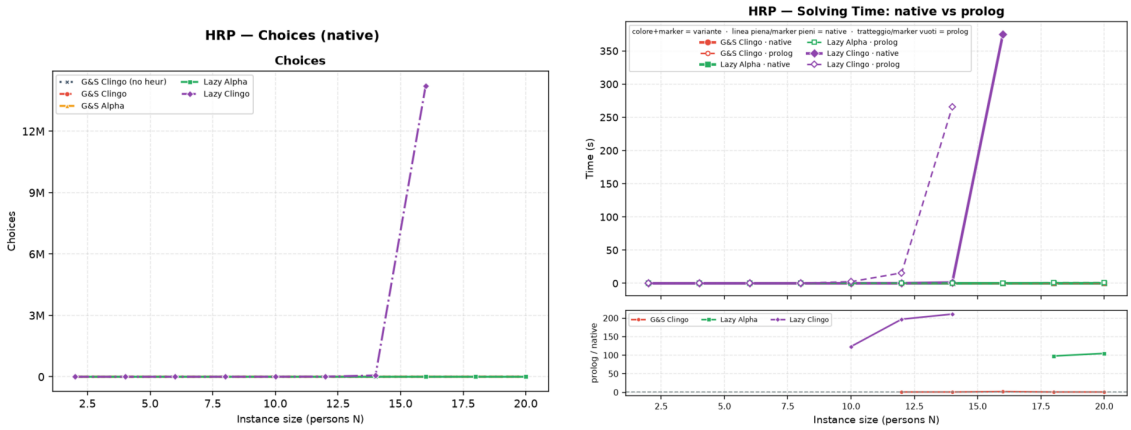

Figure 17: HRP: solver choices on the native backend (left) and the native-versus-Prolog solving comparison (right). The only diverging curve is `lc`, in both plots.

The Alpha reading behaves exactly as designed: `la` tracks `ga` decision for decision (86 versus 88 choices at $N = 14$, zero conflicts for both, and the same agreement at every size), confirming on a second problem class that the lazy propagator reproduces the intended Alpha guidance. The clingo reading, by contrast, is *actively toxic* on this heuristic: requiring established falsity on guards such as `not fullCabinet(C)` delays every level of the policy until the corresponding closure information exists, and the surviving low-level modifications steer the solver into a pathological region. Native `lc` needs 66,717 choices and 32,353 conflicts at $N = 14$ (almost eight hundred times the `la` count), and the divergence is super-exponential in N: at $N = 16$ it takes 14,218,384 choices and 8,136,557 conflicts, a factor of two hundred over the previous size, which it still just manages to close in 375.5 of the 600 available seconds; from $N = 18$ it times out. The Prolog twin is one size behind, timing out already from $N = 16$.

Two details of this comparison are worth stating precisely, because they qualify how much the frontier can be made to carry. First, the two backends do *not* reproduce identical trajectories here: at $N = 14$ native `lc` records 66,717 choices and 32,353 conflicts against the Prolog twin's 73,143 and 41,471, and similar few-percent gaps appear at every size. HRP is the family in which the heuristic produces hundreds of equally ranked candidates per decision, so the deterministic tie-break of the two implementations resolves them differently, and once a search this unstable has taken one different decision the counts drift apart. What certifies the collapse as a property of the *semantics* rather than of one engine is therefore not the identity of the counts but the fact that both backends reach the same pathological order

of magnitude, four to five orders above `la`, from encodings that differ only in the semantics token. Second, and for the same reason, the one-size gap between the frontiers is not purely per-decision cost: part of it is the different trajectory. The claim the data supports is the qualitative one — the clingo reading destroys this heuristic on both backends — and it is a strong claim precisely because it survives the change of engine. Where BSP showed the clingo reading as inert, HRP shows it as misleading: the two failure modes of "late" information are both on record.

The Prolog backend adds the cost dimension, and HRP is where it is most visible. The reference rules enumerate every applicable (cabinet, thing) and (room, cabinet) pair, more than a thousand candidates per decision at $N = 20$, so even the well-guided `la` pays 9.5 milliseconds per consultation — the most expensive query in the campaign — and spends 65% of a 1.6-second run inside the propagator, against 0.29 seconds for the native backend, whose compiled candidate scan answers the same query in 10^{-4} milliseconds. The misguided `lc` multiplies 3.5 milliseconds by 73,143 consultations: 240 of its 266 seconds at $N = 14$ are spent inside candidate-rich Prolog queries and 11 more in synchronization, 99.4% of the run, against 1.36 seconds for the native backend on the same instance. That $\times 196$ ratio is the largest in the whole campaign, and it is a measurement of the evaluation engine, not of the heuristic: the search it drives is equally bad on both sides, and only the price of asking differs.

3.13 Native versus Prolog: Summary

Across the full campaign the two backends agree on *what* is solved in every cell except four, and all four are cells in which the per-decision cost is the binding constraint: `lc` on PUP ($N = 100 \to 40$), `la_aux` on BSP ($n = 60 \to 30$), `lc` on HRP ($N = 16 \to 14$) and `lc` on BSP ($n = 160 \to 140$). Everywhere the propagator is cheap, the frontiers coincide: `la` stops at the same size on both backends on all three families.

For every ground variant the two systems run byte-identical encodings on the same solver, and their choice and conflict counts agree to the unit on all three families, which doubles as an end-to-end sanity check of the whole pipeline. For the lazy variants the native backend is faster per consultation by three to four orders of magnitude, and this is the only cross-backend comparison that is safe to make on time: total times are not, because the grounding component they are dominated by carries the node spread of Section 3.2, which is why several BSP cells report a nominally faster Prolog total (for instance 56.2 against 73.3 seconds for `la` at $n = 100$) on a run whose propagator cost 37 milliseconds in total. Where the propagator's share of the run is large, the comparison becomes meaningful again and always favours the native backend, by factors from 1.6 to 196. The Prolog backend remains the more expressive and the more observable one, which is the trade-off it was designed to embody.

3.14 Summary of Results

The full campaign supports seven conclusions. First, the lazy representation reduces the ground heuristic component from $\Theta(n^3)$ directives to two constant facts on BSP,

and the saving is confirmed on real memory wherever the heuristic grounding is a significant share of the footprint, culminating in PUP where, under a raised 600-second/30 GiB budget that both Alpha-like variants clear over the entire tested range ($N = 20$ to 200), the lazy variant costs half the time and half the peak memory of the ground heuristic simulation at every one of the ten common sizes. Peak memory is the load-bearing measure here, because it is the one the cluster's node heterogeneity cannot perturb.

Second, the total runtime does not become constant, because the base program is still fully grounded, and on BSP this is now the *only* thing that sets the frontier: `gc_noheur`, `la` and `lc` all stop between $n = 160$ and $n = 170$, and what distinguishes them there is not reach but cost, since `la` arrives at the common wall having spent 0.04 seconds of solving against the baseline's 45.3. The lazy mechanism isolates the heuristic from the growth of the instance, no more and no less; where the instance itself is the bottleneck, the correct claim is a thousandfold reduction of the search component, not a better frontier.

Third, the semantics axis is not a detail: the Alpha reading turns the BSP heuristic into a perfect greedy strategy (n choices, zero conflicts, at every size) and reproduces the intended reference policy on HRP decision for decision, while the clingo reading is inert on BSP (bit-identical to the baseline, with a maximum of zero candidates over 78,565 consultations) and actively harmful on HRP (four to five orders of magnitude more choices, on both backends independently). PUP settles the point in the cleanest possible form, because there `la` and `lc` ground literally the same program and differ by one token: 272 choices against a timeout. Representation and semantics must never be conflated.

Fourth, the faithful *ground* simulation of the Alpha reading (`ga`) is doubly punished at scale, in the grounder by the unrolling and in the solver by the unguarded activations, on BSP severely enough that it is the only heuristic variant to fail before the heuristic-free baseline does, and by nine sizes, and on PUP consistently enough that it costs twice the time and twice the memory of the lazy variant at every size both solve; this makes the lazy propagator not merely cheaper but the only practical carrier of that semantics inside clingo at scale.

Fifth, laziness is a space–time trade-off whose runtime side is now measured on both backends, not merely declared: the overhead factorizes as decide calls times per-consultation cost, the propagator's share of a run ranges from nothing (BSP `la`, 73 milliseconds on a 164-second run) to 99.4% (HRP `lc`), the cost of one consultation differs between the compiled and the interpreted backend by three to four orders of magnitude, and the total is smallest exactly when the heuristic guides well.

Sixth, the three exploratory controls delimit the approach from every side: `ga_weak` shows that the Alpha activation rule without the dynamic aggregate is behaviourally inert, `la_aux` shows that materializing the heuristic body forfeits the benefit entirely and on the Prolog side turns synchronization into 99% of the run, and `la_co` shows that pairing the lazy heuristic with a propagation-friendly model eliminates the residual ground cost and solves the entire range, $n = 200$ included, in milliseconds.

Seventh, the two backends agree on every scientific conclusion and on every frontier except the four cells where the per-decision cost is itself the binding con-

straint, so the choice between them is a genuine engineering trade-off rather than a correctness question.

4 Discussion and Conclusions

4.1 Discussion of the Results

The work shows that an important part of the cost of declarative dynamic heuristics can be moved from grounding to search. On BSP, native `#heuristic` directives grow as $\Theta(n^3)$ and exceed one million ground objects at $n = 100$; the lazy variants keep two facts, and the difference is visible in the propositional instance size, in the peak memory, and in the timeout profile. The mechanism generalizes, and the full campaign measures the generalization: on PUP, whose heuristics multiply four unrolled value domains per ground pair, the ground Alpha simulation and the lazy variant both clear the entire tested range under a raised 600-second/30 GiB budget, but the ground simulation costs twice the time and twice the memory of the lazy variant at every one of the ten common sizes, because the unrolled directives keep growing with the value domains being unrolled; both dominate the unguided baseline, which stalls by timeout at less than half their reach. Where the heuristic grounding is the binding constraint, the lazy representation converts directly into a large cost advantage over the best ground heuristic, and into solved instances relative to the unguided baseline; where the base program dominates, as at the top of the BSP range, it converts into an unchanged frontier reached with the search component reduced by three orders of magnitude, and the residual wall belongs to the domain encoding, not to the heuristic. It is worth being explicit that the BSP frontier is genuinely unchanged and not improved: the lazy variant, the clingo-like one and the heuristic-free baseline all stop within one size of each other, and the differences among them there fall inside the node-to-node spread of the cluster's grounding times. On that family the honest claim is about cost, not about reach, and the campaign's design — structural counters alongside wall-clock times — is what makes the distinction visible instead of letting a favourable node stand in for a result.

This result should not be read as a replacement for general lazy grounding. clingo still grounds the base program; the propagator acts only on the heuristic component. Nor should it be read as a claim that laziness is free: the runtime work is real, it is paid at every decision, and the per-decision metrics exist precisely to keep that cost on the record. The honest formulation is a space–time trade-off with a crossover: below it, ground-and-solve remains competitive, because the explosion is manageable and the lazy machinery (in the Prolog case, an entire in-process logic-programming engine) carries a fixed overhead; beyond it, the constant-size heuristic component is the only representation that survives.

The second central finding is semantic, and it is a finding about *meaning*, not performance. The same declarative heuristic changes behaviour dramatically depending on how partial information is read. Under the Alpha-like reading, the BSP heuristic is a perfect greedy strategy (n choices, zero conflicts) and the HRP policy reproduces its intended level structure decision for decision. Under the clingo-like reading, the very same weights and bodies degenerate in two distinct ways that the campaign now documents separately: on BSP the guards never open at decision time, the propagator reports zero applicable candidates per call, and the search is bit-identical to the baseline (inertness); on HRP the surviving low-level modifica-

tions actively steer the solver into a region almost eight hundred times larger at $N = 14$ and four orders of magnitude larger at $N = 16$, with both backends independently reaching the same pathological order of magnitude of choices even though their exact counts differ (misguidance). Neither reading is "wrong": they answer different questions, and the four-variant matrix exists to keep those questions separate. A comparison that silently mixed the axes, for example by measuring `la` against `gc` and attributing the difference to laziness, would be methodologically invalid; the thesis takes some care never to do so. The campaign adds a corollary about the ground side of the matrix: the faithful ground simulation of the Alpha reading, `ga`, is punished twice at scale, once in the grounder by the unrolling and once in the solver by the activations that its missing guards can no longer switch off, to the point that it is the only BSP heuristic variant that times out before the baseline does. Inside a ground-and-solve pipeline, the lazy propagator is thus not merely the cheaper carrier of the Alpha semantics; at scale it is the only practical one.

The third finding concerns the translation layer between the two heuristic languages. Alpha's weight–priority pairs are global levels; clingo's priorities arbitrate on a single atom. Every encoding that expresses an ordering between different atom families must therefore flatten its levels into weights wherever clingo's reading applies: in the ground variants, because clasp is the consumer, and in the lazy clingo variants, because the propagator deliberately reproduces clingo's behaviour there. This is easy to state and easy to get wrong: an early revision of the lazy-clingo PUP and HRP encodings carried Alpha-style priorities that the clingo semantics does not rank globally, producing an unintended decision order that inverted the designed one (sensors before zones; cabinet-filling before reuse), and diverging from the Prolog twin encodings of the same variants. The final audit caught and corrected the discrepancy by applying the same flattening used in the ground encodings. The episode is instructive: the interaction between priority semantics and evaluation backend is exactly the kind of silent, non-crashing behavioural divergence that a benchmark pipeline cannot detect from exit codes, and it motivates the equivalence and wiring guards that the suite now includes.

The lesson was then turned into a structural decision rather than a matter of vigilance. As long as the priority field is used to carry levels in *some* encodings, its meaning depends on the pair (semantics, backend): a global level under `__semantics(alpha)` in the native propagator, a purely local arbiter under `__semantics(clingo)` in the same propagator, and a global level again in the Prolog backend, whose selection rule sorts by (priority, weight) regardless of the semantics selector. Three readings of one syntactic field is a standing invitation to the very bug just described. Since flattening is order-preserving, the suite now removes the ambiguity by construction: the levels always live in the weight, in every variant and in both backends, and the priority field is reserved for local arbitration on a single atom. Two properties follow. First, the encodings of a pair differ by exactly one token, so the axis under study is the only thing that varies. Second, the change is behaviour-preserving with respect to the earlier revision: on every benchmark the flattened weights reproduce the previous total order exactly, because on each instance the level multiplier strictly dominates the intra-level weights, so the experimental results reported here are unaffected. The native global-level ranking remains implemented and unit-tested; comparing it

against its flattened equivalent, on encodings where the two are no longer required to coincide, is a controlled experiment left to future work.

A related engineering lesson comes from the silent-fallback failure mode of the Prolog backend. A single missing environment variable degrades every lazy Prolog run to the no-heuristic baseline without any error, since the fallback evaluator simply fails on the Prolog-style rule bodies. The pipeline defends itself with positive evidence of activity (`decide_calls > 0`), cross-backend agreement checks, and a wrapper that forces the variable; the general principle, that a heuristic mechanism must prove it ran rather than merely not fail, seems worth stating explicitly.

Finally, the auxiliary-form experiment delimits the approach from within. `la_aux` shows that laziness only pays if the *entire* dynamic part of the heuristic, aggregate included, stays out of the grounder: materializing the heuristic body into an auxiliary relation reintroduces the very product the mechanism was built to avoid.

4.2 Limitations

The current implementation has explicit technical limitations. The native backend indexes targets and bodies through numeric tuples only; non-numeric arguments are not supported. The arithmetic language covers addition, subtraction, and multiplication; the available dynamic aggregates are sum, count, min, and max, over a single source predicate, with joins delegated to precompiled auxiliary predicates in the encoding. The modifiers `init` and `factor` are recognized but deliberately rejected: in native clingo they modify solver-internal scores that the `decide` interface cannot reach, and approximating them as `level` would silently change their meaning. The propagator is registered sequentially and has no synchronization layer for parallel solving. The tie-break between targets of equal rank is deterministic but does not reproduce clingo's internal scores.

On the evaluation side, the campaign now covers all three families on both backends with the corrected encodings, and both backends are phase-timed, but its scope must be stated precisely. The most important limitation is one of measurement rather than of design: jobs are dispatched by SLURM across heterogeneous nodes, and the grounding time of one and the same program varies between them by a median of 13% and by up to 52%. Every time-based claim in this thesis is therefore qualified by that spread, and only the claims whose margin is an order of magnitude larger — the factor of two between `ga` and `la` on PUP, the two orders of magnitude of `lc` on HRP, the three orders of magnitude on the BSP solving component — are load-bearing. The structural counters, which are exact, carry the rest. A campaign with repeated runs per cell, or pinned to a homogeneous partition, would let the BSP frontier itself be claimed rather than only the cost behind it; this was not affordable within the cluster budget and remains the single most valuable extension of the experimental protocol.

The HRP instances remain easy for clasp at every benchmarked size, so HRP supports conclusions about guidance quality and per-decision cost, not about frontiers. The PUP frontier claims rest on one instance per size (the `double` family): the non-monotone behaviour observed for `gc` and `lc` is a reminder that neighbouring sizes differ in hardness, and per-size variance is not measured. The two backends also

do not reproduce identical search trajectories on PUP and HRP, where the heuristic ranking admits ties that the C++ and Prolog tie-breaks resolve differently; the divergence is a few percent on `la` and larger on the unstable `lc` searches, which is why the cross-backend comparisons in those families are read qualitatively rather than as controlled measurements of the engine alone. The memory metric is end-to-end process RSS, which is the honest but blunt instrument discussed in Sections 2 and 3: it includes the Prolog engine for the lazy Prolog variants and cannot separate grounding memory from search memory. The auxiliary clingo evaluator of the Prolog build re-grounds a program per decision and is a functional reference, not a performance-comparable backend. Finally, the whitelist of static predicates and the inference of the program constant n in the Prolog backend are benchmark-specific conveniences that a production version should replace with a declarative interface.

4.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose between fully grounded `#heuristic` directives and procedural heuristics hard-coded in the solver: a compact declarative surface, evaluated at runtime with access to the partial assignment, is a workable third point in the design space, and it can host more than one evaluation semantics behind one syntax. The two backends demonstrate two implementation strategies for that layer, a compiled, incremental one and an embedded general-purpose engine, with the expected trade-off between per-decision efficiency and expressive freedom of the rule bodies.

From an engineering perspective, the work also shows that clingo's propagator and heuristic interfaces are flexible enough to prototype non-trivial guidance mechanisms without touching the solver core: watched literals, incremental aggregate states, per-target modifier resolution, and a global decision ranking are all expressible against the public API of `Clingo::Heuristic`.

4.4 Future Work

Several extensions follow naturally. The mechanism should become configurable rather than always-on, and the expression language deserves integer division, modulo, and comparisons, together with a principled treatment of non-numeric symbols through a stable internal mapping. Aggregates with in-template joins would remove the need for precompiled auxiliary predicates. On the Prolog side, the synchronization protocol could move from per-literal goals to batched updates, and the backend could expose its per-decision statistics through clingo's statistics interface instead of stderr parsing. A parallel-solving synchronization layer and a tie-break policy that cooperates with the solver's own scores are natural solver-side improvements.

On the evaluation side, the PUP synchronization numbers make the batched-update optimization concrete and measurable: at `double-160` the trail mirroring costs eleven times the queries it serves (5.7 seconds against 0.5), and at `double-200` ten times (16.1 against 1.6), while the native backend performs the same task in 0.07 seconds; a protocol that coalesces retract–assert pairs per propagation batch

therefore has both a quantified target to beat and an existence proof that the target is reachable. Beyond that, configuration, scheduling, and packing domains are natural candidates for broadening the evaluation, precisely because their useful heuristics are dynamic; PUP would additionally benefit from multiple instances per size (the `doubleV` family) to put variance bars behind the frontier claims. A further direction is a verified converter from annotated `#heuristic` directives to lazy facts, with the flattening rule of Section 2.1 applied mechanically rather than by hand; the audit episode discussed above is a concrete argument for automating that step.

4.5 Conclusions

This thesis presented a lazy evaluation mechanism for declarative domain-specific heuristics in clingo, instantiated in two backends over a common propagator design: a native C++ backend interpreting compact `__heuristic` facts with incremental dynamic aggregates, and an in-process SWI-Prolog backend evaluating heuristic rules as logic programs against a synchronized image of the solver trail. The mechanism supports two explicit readings of partial information, the clingo-like and the Alpha-like semantics, and reproduces clingo's weight, priority, and modifier behaviour where that reading applies, including the systematic flattening of Alpha's global priority levels into clingo weights.

The full campaign on BSP, PUP, and HRP under uniform limits (600 seconds, 30 GiB) shows a reduction of the ground heuristic component from $\Theta(n^3)$ directives, more than one million at $n = 100$, to two constant facts, with the corresponding saving in real memory wherever the heuristic grounding matters; on PUP the lazy variant reaches more than twice the solvable frontier of the unguided baseline and costs half the time and half the memory of the best ground heuristic at every size both solve, while on BSP, where the base program owns the frontier, it removes the search cost outright — n choices and zero conflicts at every size, against tens of thousands for every alternative — and reaches the same wall as the baseline having spent 0.04 seconds of solving where the baseline spends 45. The campaign also quantifies the price, on both backends: a per-decision runtime cost that ground-and-solve does not pay, which factorizes into consultation count times consultation cost, ranges from nothing to 99% of a run, differs between the compiled and the interpreted evaluator by three to four orders of magnitude per consultation, vanishes when the heuristic guides well, and dominates when it does not. The suite validates that all variants and both backends preserve the answer sets and guards against silent misconfiguration.

The final result is a working, documented, and measurable prototype together with a methodology: two orthogonal axes that keep representation and semantics separate, a flattening rule that makes Alpha-style encodings meaningful under clingo's reading of priorities, and an experimental discipline that states what each number measures. It does not solve the grounding bottleneck in general, but it provides a concrete, verifiable path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

[1] Michael Gelfond and Vladimir Lifschitz. "The Stable Model Semantics for Logic Programming". In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.

[2] Martin Gebser et al. *Answer Set Solving in Practice*. Morgan and Claypool Publishers, 2012.

[3] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: `https://potassco.org`.

[4] Martin Gebser et al. "Domain-Specific Heuristics in Answer Set Programming". In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.

[5] Carmine Dodaro et al. "Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems". In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.

[6] Antonius Weinzierl. "Blending Lazy-Grounding and CDNL Search for Answer-Set Solving". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.

[7] Richard Comploi-Taupe et al. "Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach". In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.

[8] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. "Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems". In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023.

[9] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.

[10] Martin Gebser et al. "clasp: A Conflict-Driven Answer Set Solver". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.

[11] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference*. 2001.

[12] Markus Aschinger et al. “Optimization Methods for the Partner Units Problem”. In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.

[13] Gerhard Friedrich et al. “(Re)configuration using Answer Set Programming”. In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.

[14] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[15] Armin Biere. *runlim: a tool for sampling time and memory usage of a process*. `https://github.com/arminbiere/runlim`. Accessed 2026.

Acknowledgements

I would like to thank Prof. Carmine Dodaro for his guidance at the University of Calabria, and Prof. Torsten Schaub and Dr. Javier Romero for their supervision and support during my Erasmus period at the University of Potsdam. Their feedback helped shape both the modelling choices and the implementation decisions behind this work.

I am also grateful to the ASP and Potassco communities. Tools such as clingo make it possible to experiment with advanced research ideas while relying on a solid, well documented, and open foundation.

Finally, I thank my family and the people who stayed close to me throughout this thesis work, for their constant support and patience during the many long debugging sessions.